

Lecture 10

Finish Linear Regression & Regularization

Assigned reading:

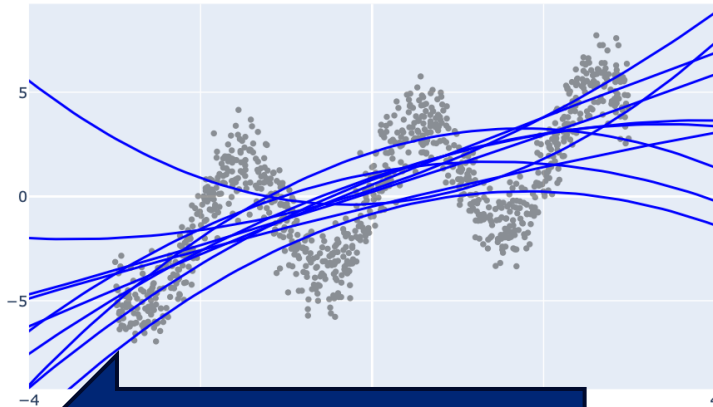
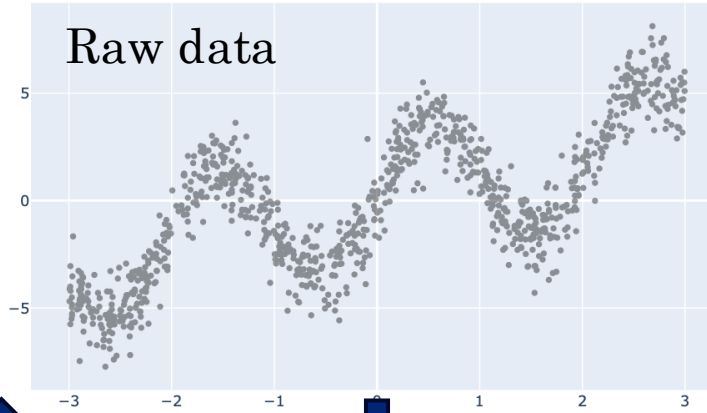
- 9.2.2 (Lasso regularization)

Roadmap

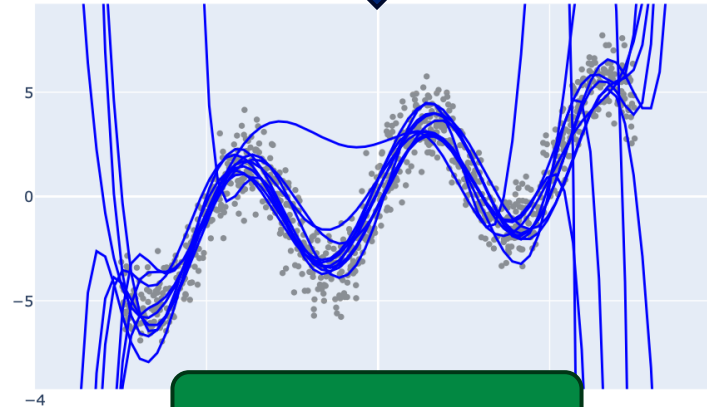
- Regularization & Linear Regression
 - Train/test/validation splits.
 - Error metrics to use.
 - Lasso regression (L_1)
- Intro to Classification

Complexity and Overfitting

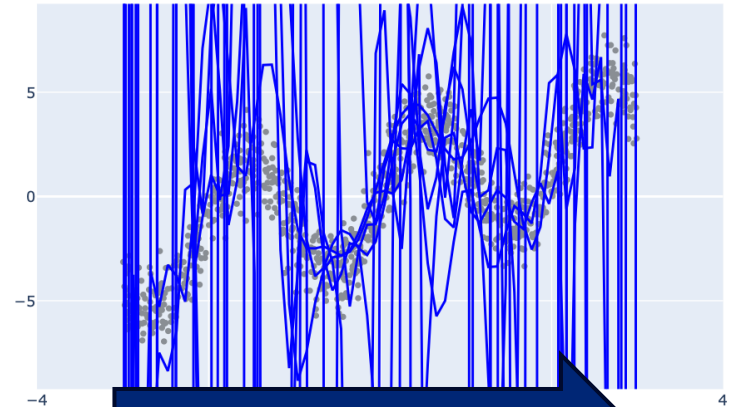
Regularization is the process of adding **constraints** or **penalties** to the learning process to **improve generalization**



Underfitting



“Sweet Spot”

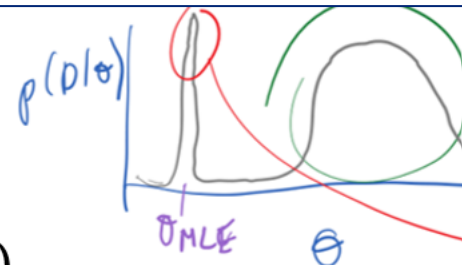


Overfitting

More Regularization

Recap from last class:

The Bayesian modelling approach



- Bayesians put a *prior distribution* on the parameters, $p(\theta)$.
- Then they seek to compute the *posterior distribution*, $p(\theta|D)$.
- Then, predictive distribution is given by

$$p(y|x) = \int_{\theta} p(y|x, \theta) p(\theta|D) d\theta = E_{\theta}[p(y|x, \theta)]$$

- Procedurally, this is done using Bayes' rule:

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}.$$

- Difficult in practice! $p(D) = \int_{\theta} p(D, \theta) d\theta = \int_{\theta} p(D|\theta)p(\theta) d\theta$
- We will be lazy, instead being fake Bayesians, yielding L2 regression:
- $\theta_{\text{lazy}} = \underset{\theta}{\operatorname{argmax}} p(\theta|D)$ Maximum A Posteriori (MAP) estimation.

Recap from last class:

Obtaining the MAP/L2 solution

$$\begin{aligned}w_{L_2} &= \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda \|w\|_2^2 \\&= \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda w^T w\end{aligned}$$

Take partial derivative and set to zero:

$$\nabla_w \mathcal{L}_{MAP} = -2A^T y + 2A^T A w + 2\lambda I w$$

$$\rightarrow 0 = -A^T y + A^T A w + \lambda I w$$

$$\rightarrow A^T y = (A^T A + \lambda I) w$$

$$\rightarrow (A^T A + \lambda I)^{-1} A^T y = w$$

$$\text{So } w_{L_2} = (A^T A + \lambda I)^{-1} A^T y.$$

If $\lambda > 0$, we can invert $(A^T A + \lambda I)$.

$$\begin{aligned}Z &= \Phi D \Phi^T \\Z^{-1} &= \Phi D^{-1} \Phi^T \\ \text{where } D^{-1} &= \begin{bmatrix} \lambda_1^{-1} & & \\ & \lambda_2^{-1} & \\ & & \ddots \\ & & & \lambda_n^{-1} \end{bmatrix}\end{aligned}$$

$\lambda > 0$ increases numerical stability even when $A^T A = QDQ^T$ is invertible:

- Let $\text{diag}(D) = \sigma_1, \dots, \sigma_d$ with $\sigma_i > \sigma_{i+1}$
- Condition # of $A^T A$ is $\kappa = \frac{\sigma_{\max}}{\sigma_{\min}}$.
- $1 \leq \kappa \leq \infty$, 1 is maximally well-behaved (robust to perturbations)
- When is $\kappa = \infty$?

- Then $A^T A + \lambda I = Q(D + \lambda I)Q^T$
- Then $\text{diag}(D + \lambda I) = \sigma_1 + \lambda, \dots, \sigma_d + \lambda$
- Condition # of $A^T A + \lambda I$ is $\kappa = \frac{\sigma_{\max} + \lambda}{\sigma_{\min} + \lambda} < \kappa$
- What happens $\lambda \rightarrow \infty$?

So $w_{L_2} = (A^T A + \lambda I)^{-1} A^T y$.

If $\lambda > 0$, we can invert $(A^T A + \lambda I)$.

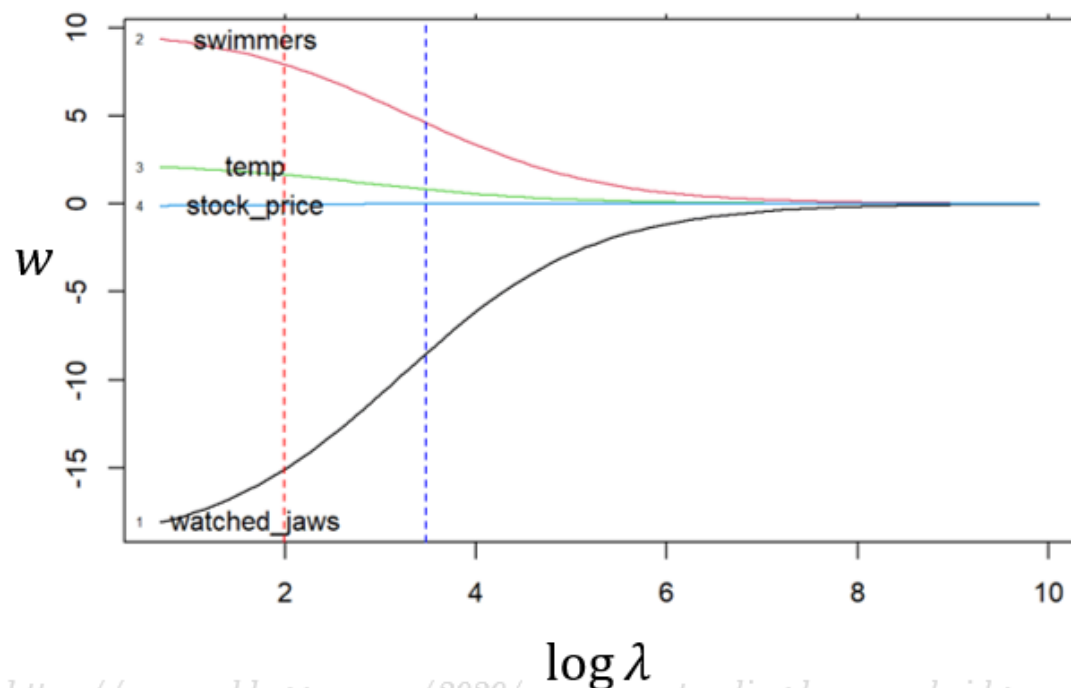
$$z = \varphi D^{-1} \varphi^T$$

where $D^{-1} = \begin{bmatrix} \lambda_1^{-1} & & \\ & \lambda_2^{-1} & \\ & & \ddots \\ & & & \lambda_n^{-1} \end{bmatrix}$

Recap from last class:

How to pick the value of λ

$$\mathcal{L}_{MAP} = (y - Aw)^T (y - Aw) + \lambda \|w\|_2^2$$



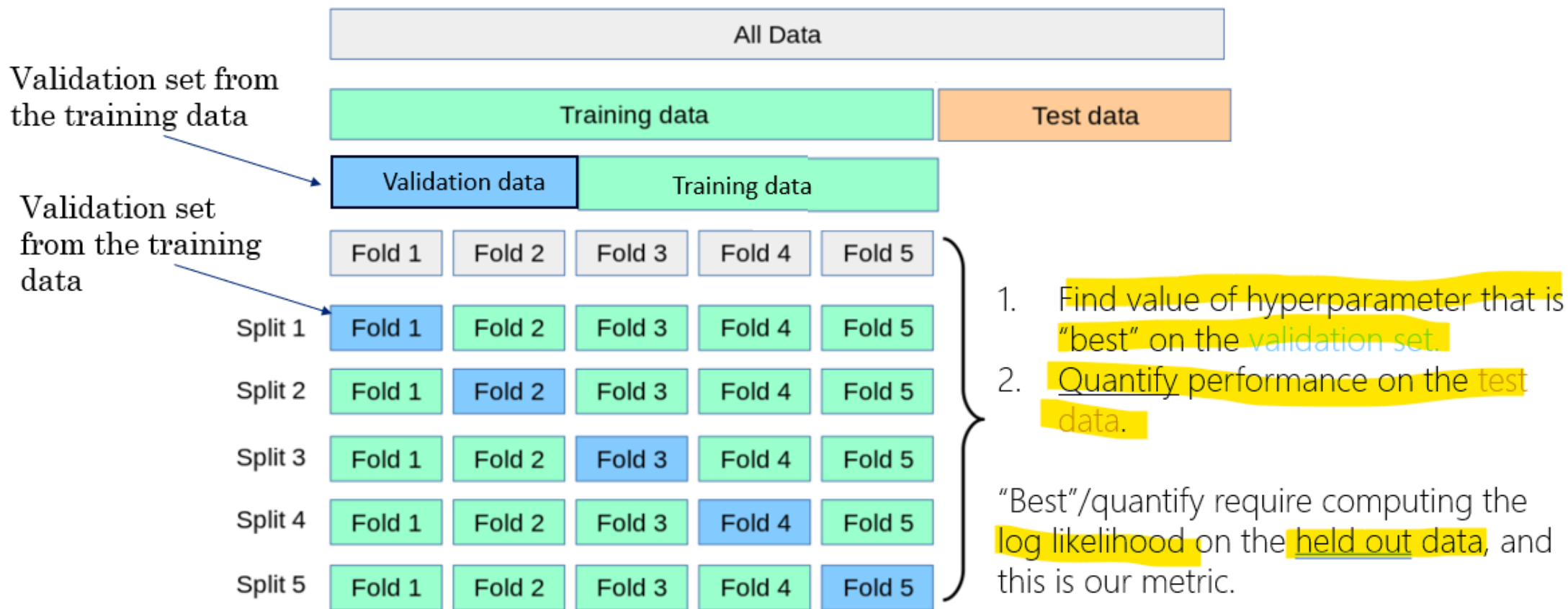
- Practically, how should we set λ ?
- Can we treat it as a parameter in the loss, and minimize wrt it?
- No: cannot use MLE!
- Need independent data, a *validation set* on which to evaluate the loss.

Roadmap

- Regularization & Linear Regression
 - Train/test/validation splits.
 - Error metrics to use.
 - Lasso regression (L_1)
- Intro to Classification

Recap from last class:

K-fold cross-validation

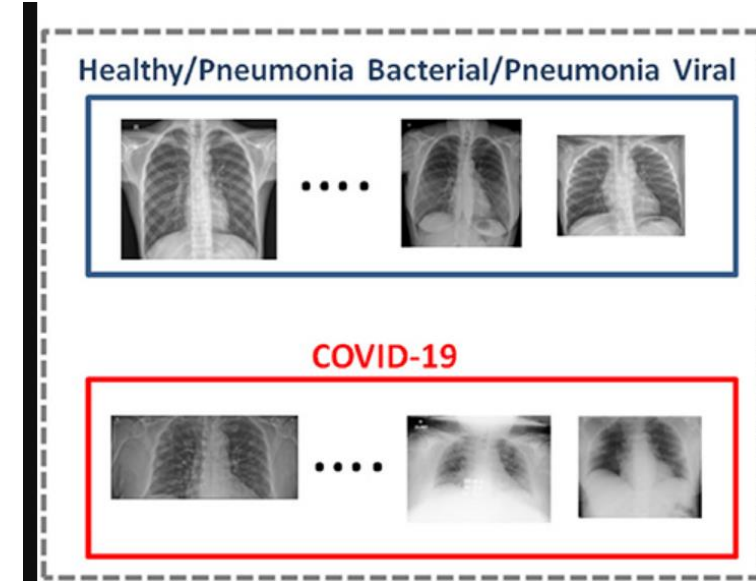


Use of train vs test sets



Always need to do *model selection*:

- What model class to use? Linear regression, neural network?
 - Picking which features to use in linear regression.
 - Picking which ridge penalty (*hyperparameter*) to use, λ .
 - For neural networks—picking what architecture to choose.
 - *Model selection* cannot be done by MLE/optimization.
- We must use train/test/validation splits to choose the *hyper-parameters*

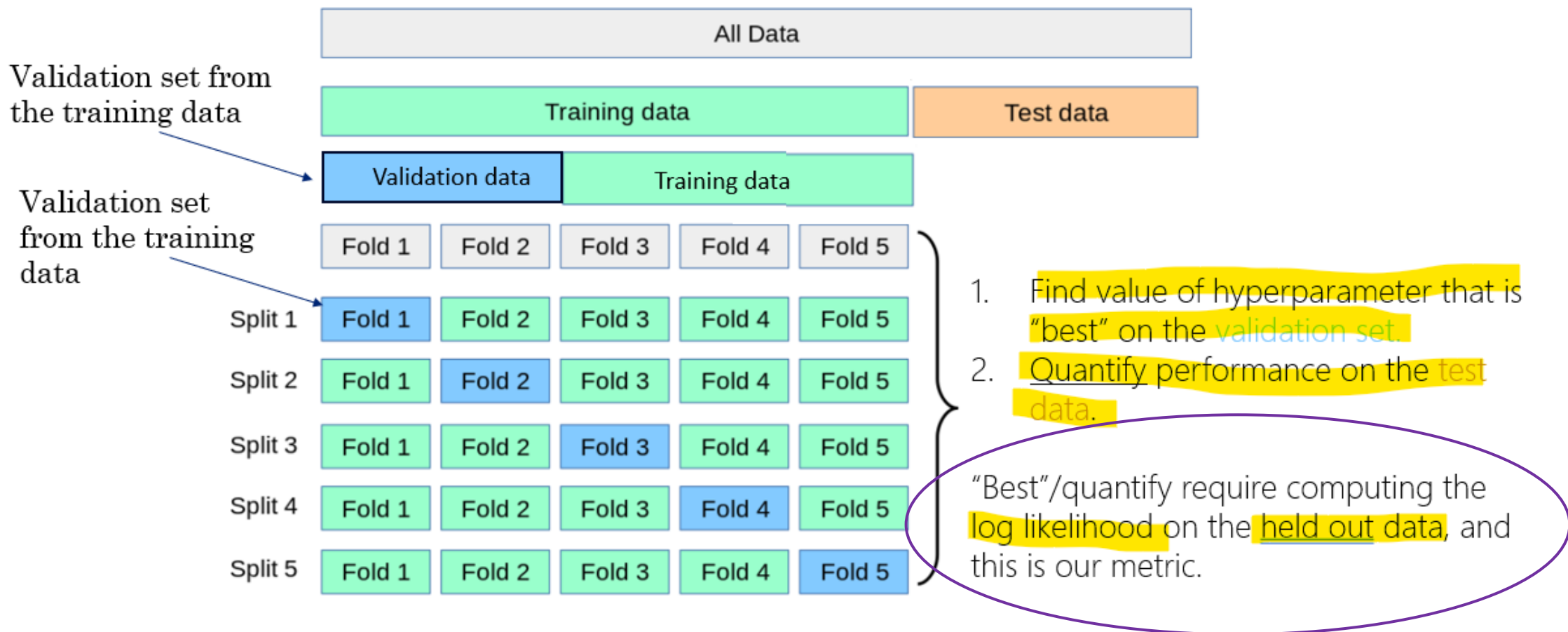


Role of train vs test (e.g. Hospital xray prediction)

- Use validation set to pick model and/or hyper-parameters.
- Through “winner’s curse” will get lower error estimate on validation set than is real.
- After picking model & hyper-parameters, use test set ONCE to get unbiased error estimate.

Recap from last class:

K-fold cross-validation



Roadmap

- Regularization & Linear Regression
 - Train/test/validation splits.
 - Error metrics to use.
 - Lasso regression (L_1)
- Intro to Classification

Evaluation – General Regression Metrics

Log likelihood

$$\frac{1}{N} \left(\sum_{n=1}^N \log p(t_n | \mathbf{x}_n, \mathbf{w}) \right)$$

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

r (correlation, also, ρ)

$$\frac{\sum_{n=1}^N (t_n - \bar{t}_n)(y(\mathbf{x}_n, \mathbf{w}) - \bar{y}(\mathbf{x}_n, \mathbf{w}))}{\sum_{n=1}^N (t_n - \bar{t}_n)^2 (y(\mathbf{x}_n, \mathbf{w}) - \bar{y}(\mathbf{x}_n, \mathbf{w}))^2}$$

R-Squared (R^2)

$$r^2$$

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(\mathbf{x}_n, \mathbf{w})| \right)$$

Evaluation – General Regression Metrics

Log likelihood

$$\frac{1}{N} \left(\sum_{n=1}^N \log p(t_n | \mathbf{x}_n, \mathbf{w}) \right)$$

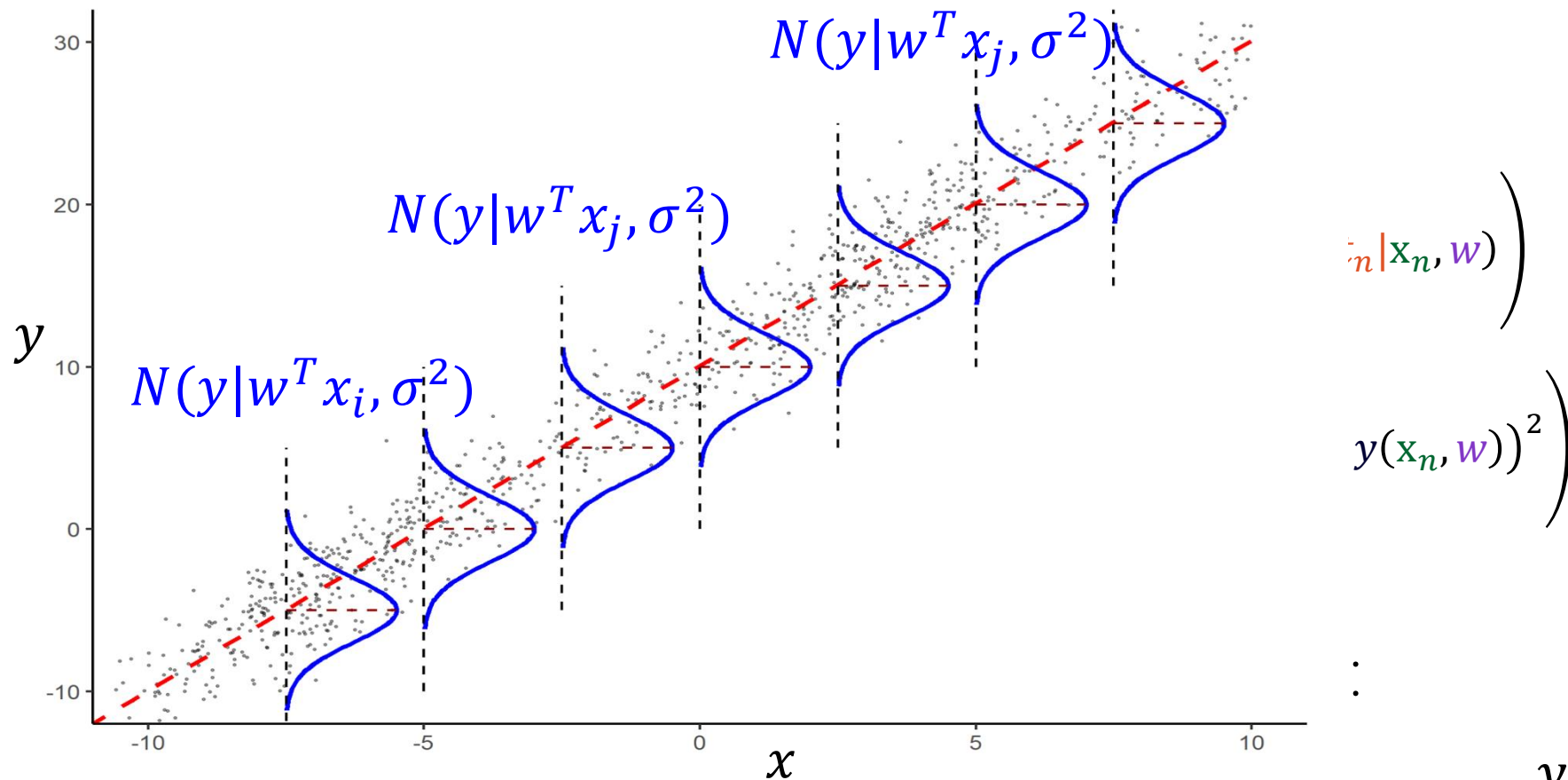
Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Thought experiment with data set, $D = \{(\mathbf{x}_i, y_i)\}$:

- Suppose I get the exact same MSE using model 1 and model 2, does this mean they are equally good?
- What might yield a more nuanced assessment?

$$y(\mathbf{x}_n, \mathbf{w}) = E(y) = \mathbf{w}^t \mathbf{x}$$



Log-likelihood provides a richer measure of model fit by evaluating how likely the entire observed dataset is under the model's full probabilistic assumptions

$$y(\mathbf{x}_n, \mathbf{w}) = E(y) = \mathbf{w}^T \mathbf{x}$$

$$\frac{1}{N} \left(\sum_{n=1}^N \log p(\mathbf{t}_n | \mathbf{x}_n, \mathbf{w}) \right)$$

$$\frac{1}{N} \left(\sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

- Suppose I get the exact same MSE using model 1 and model 2, does this mean they are equally good?
- What might yield a more nuanced assessment?

Roadmap

- Regularization & Linear Regression
 - Train/test/validation splits.
 - Error metrics to use.
 - Lasso regression (L_1)
- Intro to Classification

Recall: Ridge as “lazy Bayesian” with normal prior

$$\begin{aligned} w_{MAP} &= \operatorname{argmax}_w \log p(D|w) p(w) = \operatorname{argmax}_w \log p(D|w) + \log N(w; 0, \lambda I) \\ &= \operatorname{argmin}_w (y - Aw)^T (y - Aw) + \lambda' \|w\|_2^2 \quad \text{for } \lambda' = \frac{\sigma^2}{\lambda}. \end{aligned}$$

Equivalence between MAP w Gaussian prior and L2 regression!

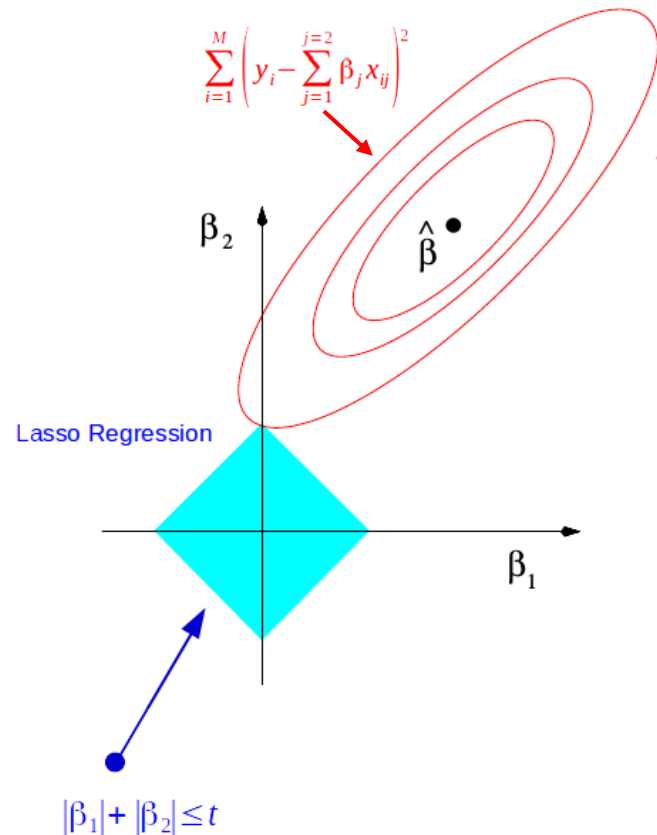
What if wanted to have the linear function only depend on a few features (*i.e.*, the coefficients of w are sparse)?

Other priors for MAP in linear regression?

- What if wanted to have the linear function only depend on a few features (*i.e.*, the coefficients of $\mathbf{w}$ are sparse)?
- Add a penalty that counts the # of non-zero weights—an L_0 penalty, $\lambda \|\mathbf{w}\|_0$.
- Not differentiable! (becomes combinatorial optimization—nasty).
- But...the L_1 norm penalty, $\lambda \|\mathbf{w}\|_1 = \lambda \sum_d |w_d|$, tends to induce sparse $\mathbf{w}$ —differentiable everywhere except at $\mathbf{w}_i = \mathbf{0}$ which can be worked around.
- This is called *Lasso* regression, or L_1 -penalized regression.

L_1 -penalized linear regression, aka *Lasso*

- $w_{L_1} = \underset{w}{\operatorname{argmin}} (y - Aw)^T (y - Aw) + \lambda \|w\|_1$
- Why does the L_1 norm penalty tends to induce sparse w ?
- Equivalent to $\underset{w \text{ s.t. } \|w\|_1 < C(\lambda)}{\operatorname{argmin}} (y - Aw)^T (y - Aw)$.
- "Pointy" constraint surface is jutting out along the axes.
- In many cases, the L_1 norm constraint will cause the unconstrained solution to intersect the constraint at a corner.
- The corners are where some coefficients are 0, which is a sparse solution..



The p -norm of a vector $\mathbf{x} \in \mathbb{R}^d$ is defined as:

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^d |x_i|^p \right)^{1/p}$$

for $p \geq 1$.

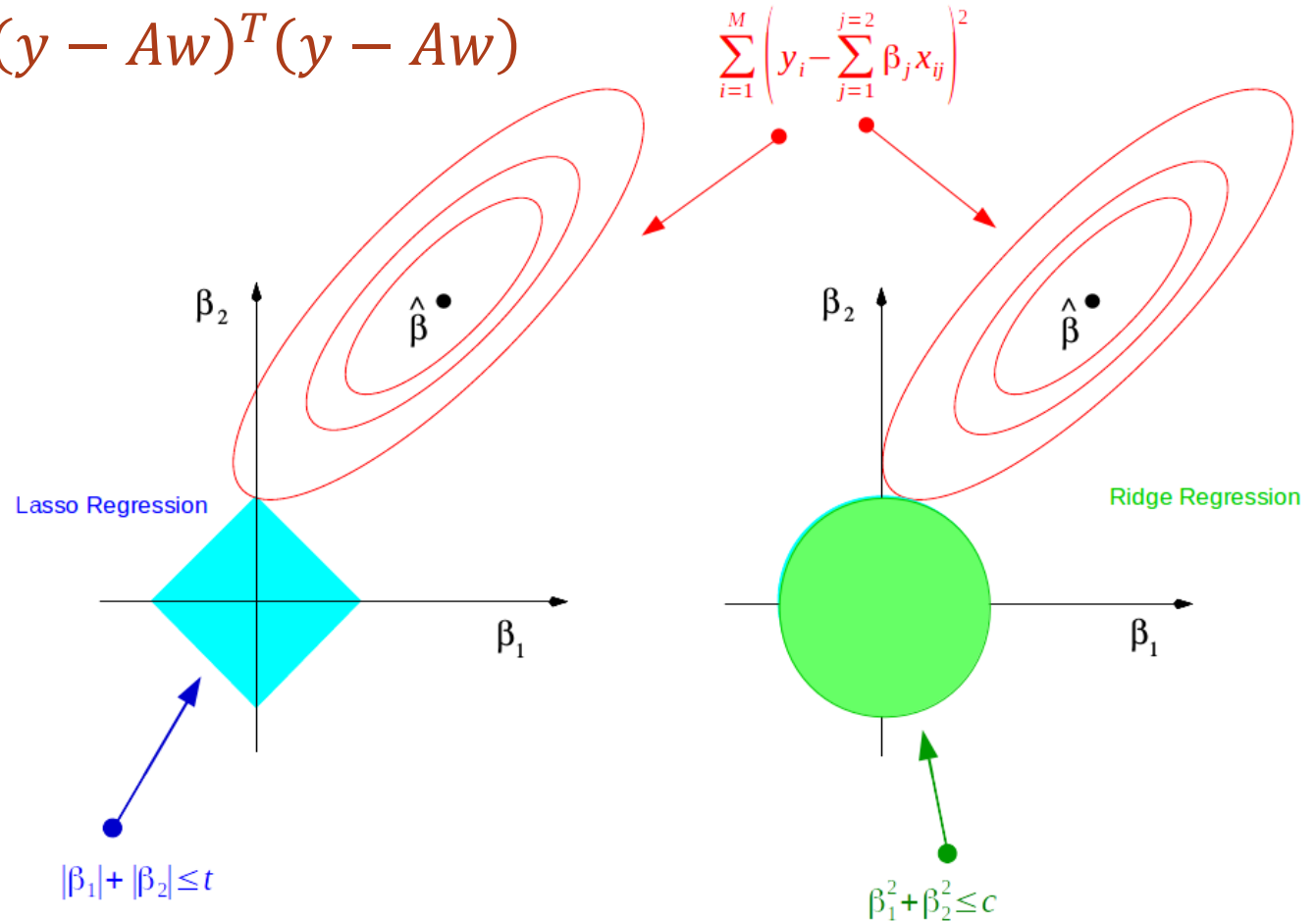
Common cases:

- $p = 1$ (Manhattan/taxicab norm): $\|\mathbf{x}\|_1 = |x_1| + |x_2| + \dots + |x_d|$
- $p = 2$ (Euclidean norm): $\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2}$
- $p = \infty$ (max norm): $\|\mathbf{x}\|_\infty = \max_i |x_i|$

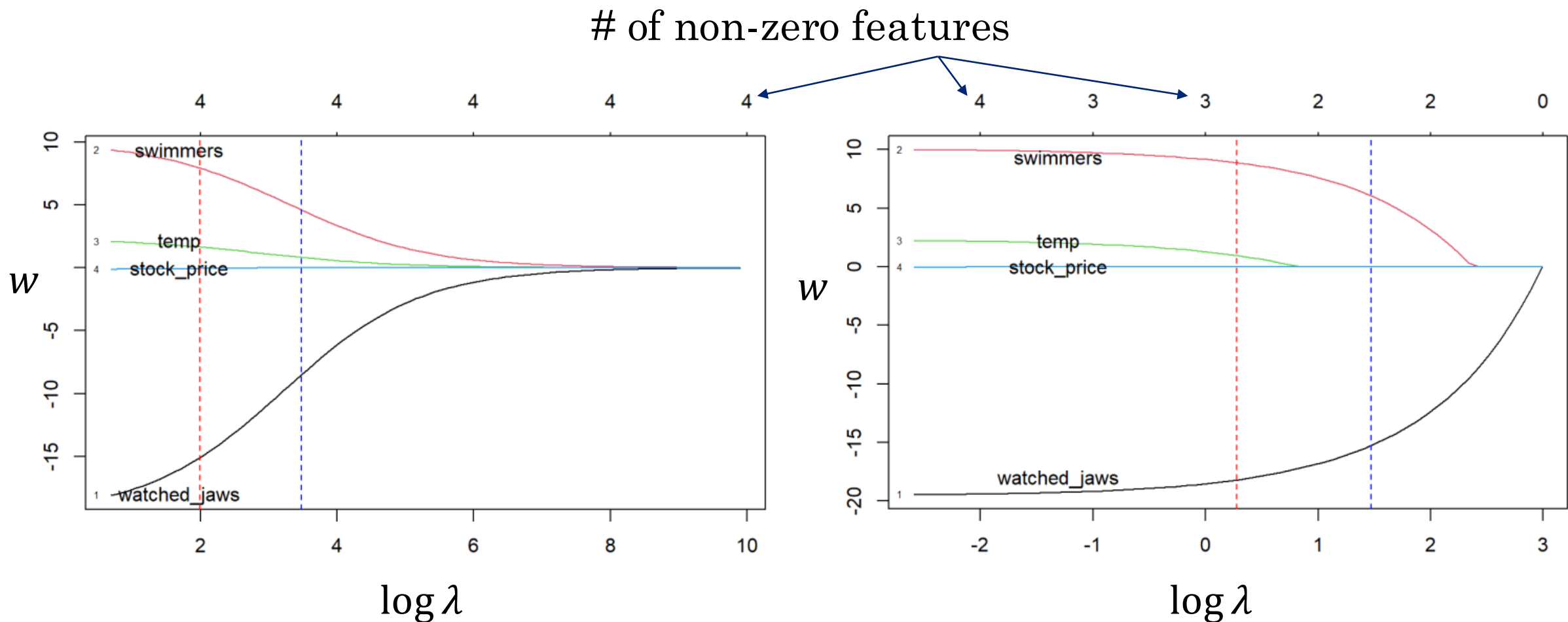
The 2-norm is what we typically mean when we just write $\|\mathbf{x}\|$ without a subscript.

L_1 -penalized linear regression, aka *Lasso*

- $w_{L_1} = \underset{w}{\operatorname{argmin}} (y - Aw)^T (y - Aw) + \lambda \|w\|_1$ $w_{L_2} = \underset{w}{\operatorname{argmin}} (y - Aw)^T (y - Aw) + \lambda \|w\|_2^2$
- Why does the L_1 norm penalty tends to induce sparse w ?
- Equivalent to $\underset{w \text{ s.t. } \|w\|_1 < C(\lambda)}{\operatorname{argmin}} (y - Aw)^T (y - Aw)$
- "Pointy" constraint surface is jutting out along the axes.
- In many cases, the L_1 norm constraint will cause the unconstrained solution to intersect the constraint at a corner.
- The corners are where some coefficients are 0, which is a sparse solution..



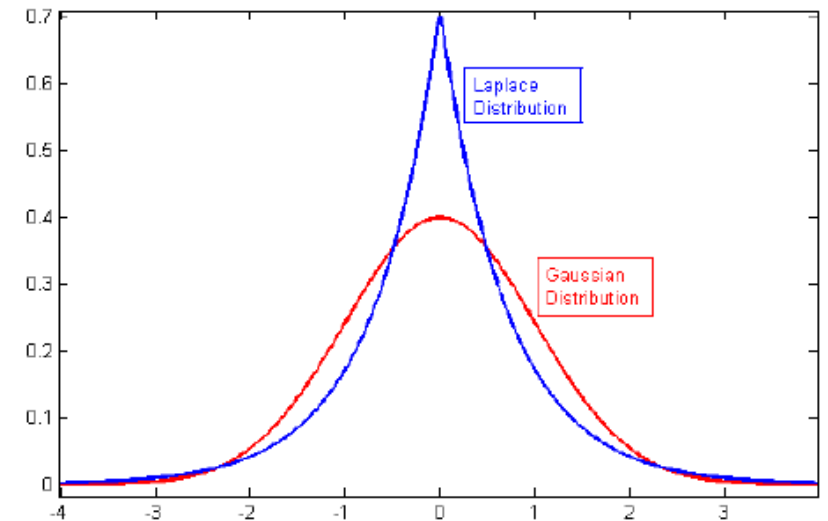
Ridge vs Lasso: shrinkage vs sparsity



MAP interpretation for Lasso/ L_1 -penalized linear regression?

- L_2 regression arose from a $N(0, \lambda I)$ prior.
- Is there a prior corresponding to L_1 ?
- Yes, the Laplace prior!

$$p(w) = \frac{1}{2\lambda'} \exp\left(-\frac{\|w\|_1}{\lambda'}\right).$$

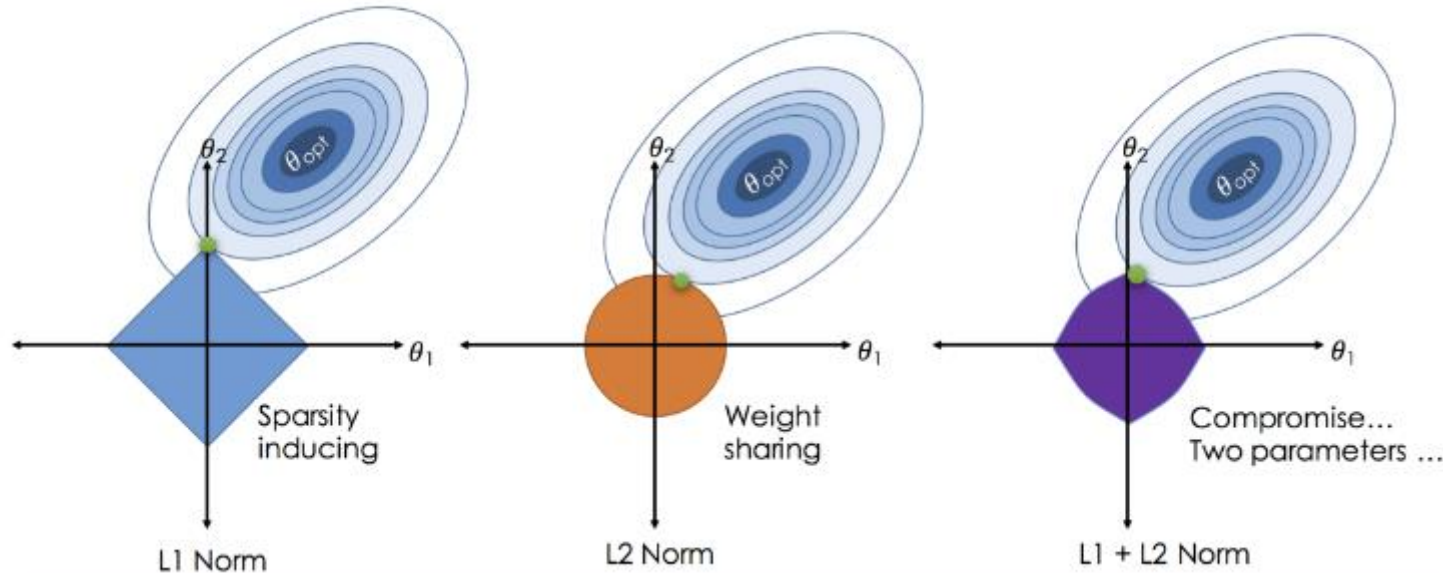


Issues with LASSO:

- If highly correlated features, tends to ignore all but one.

Combine L_1 and L_2 penalties?

Yes, “elastic net regression”.



What makes this much more computationally expensive to use than either L_1 or L_2 alone?

Aside: a thought experiment on regression

Consider:

- Use MLE on data, $D = \{x_i, y_i\}$ to get $\hat{y}_i = p_{\theta}(y|\Phi(x))$ using linear regression.
- Assume an abundance of data (millions of data points), and only 100 parameters.
- Suppose get accuracy $\mp \$1000$ of sale price when applying to held out part of our data.
- Can we assume this model will get $\mp \$1000$ on any test set that may come in the future?

Actual vs. predicted sale price of house

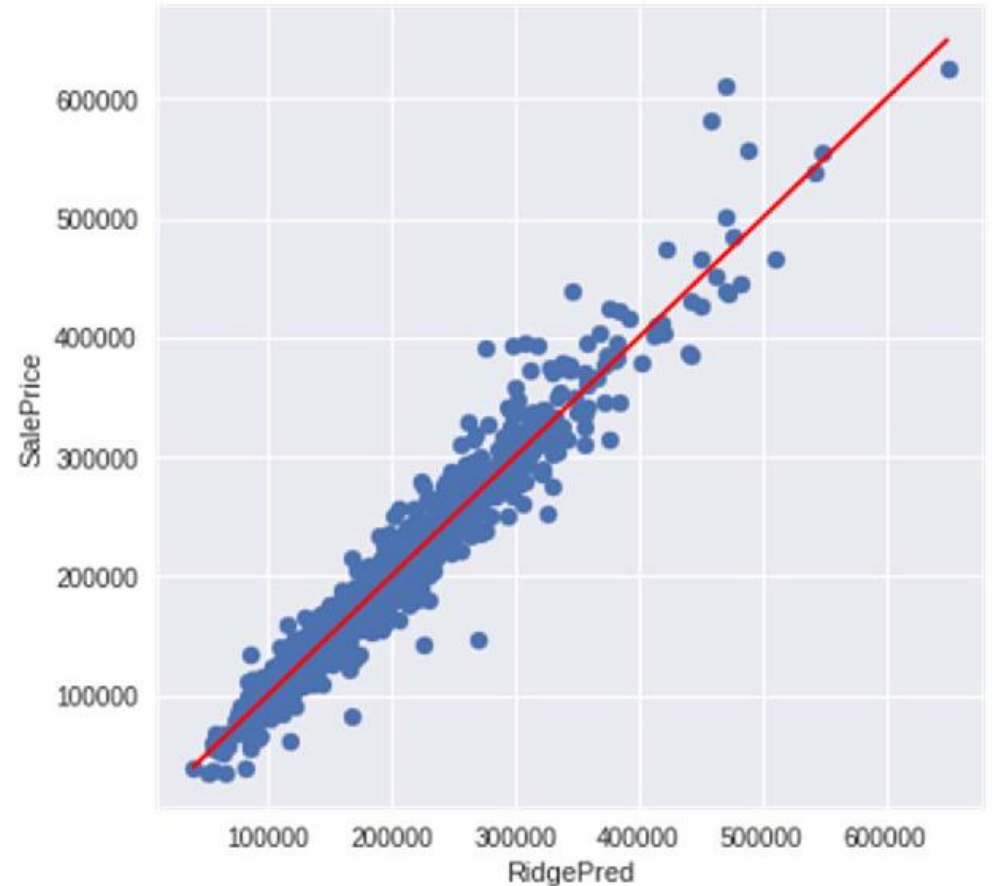


Fig. 4 Ridge Prediction for Training Data.

Causation vs correlation

Breakingviews

Zillow's failed house flipping

Reuters

WSJ NOV. 2021 : *“The company expects to record losses of more than \$500 million from home-flipping by the end of this year and is laying off a quarter of its staff.”*

Actual vs. predicted sale price of house

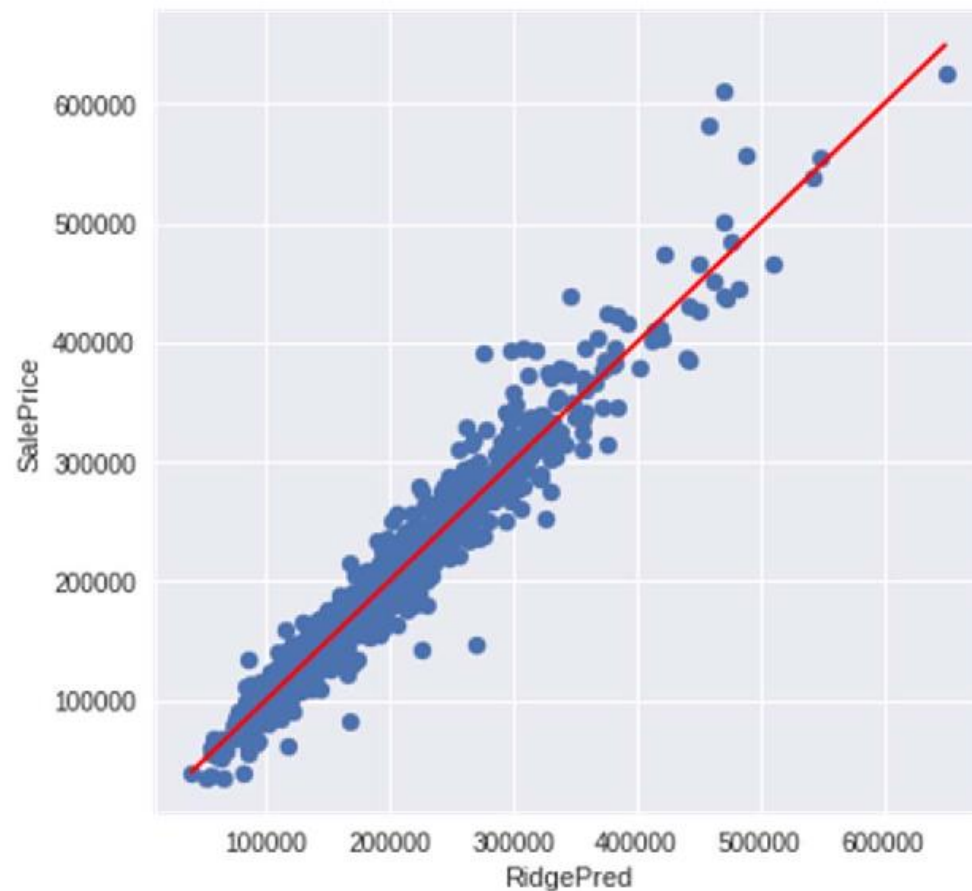


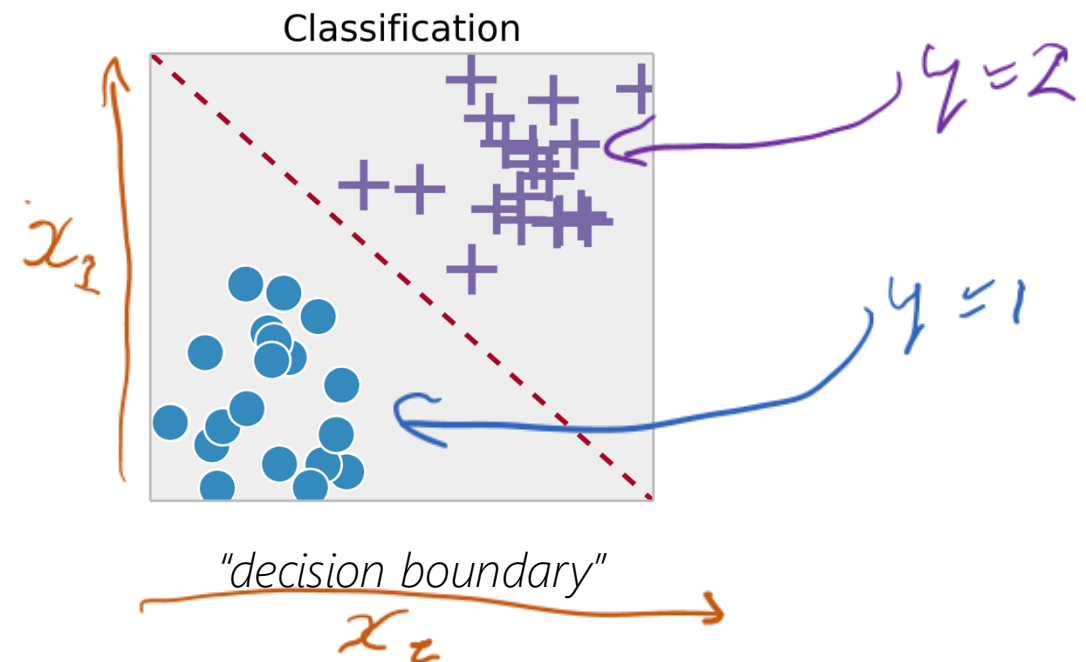
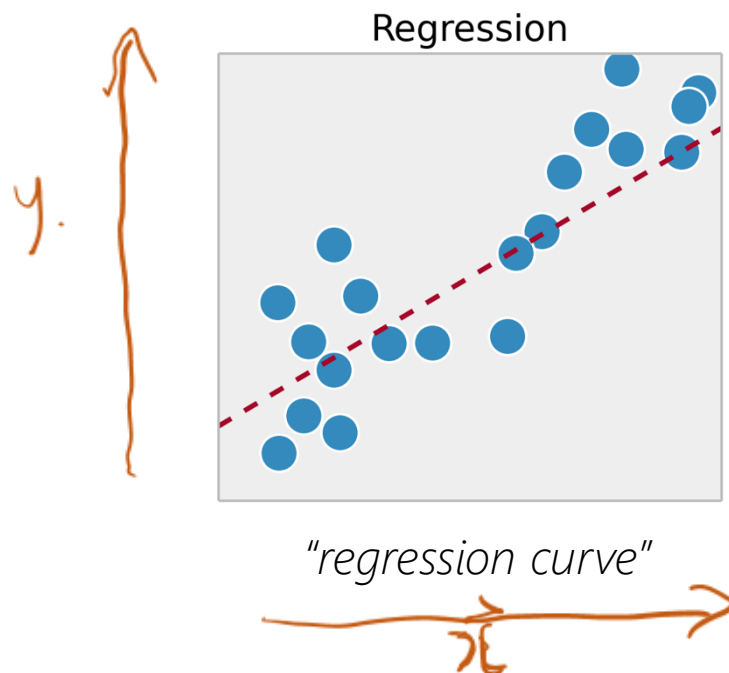
Fig. 4 Ridge Prediction for Training Data.

Roadmap

- Regularization & Linear Regression
 - Train/test/validation splits.
 - Error metrics to use.
 - Lasso regression (L_1)
- Intro to Classification

Recall: classification vs. regression

- Regression: try to “draw a line to trace out the data”.
- Classification: try to “draw a line to separate the classes of data”.
 - i.e. separate with a *decision boundary*: further away point is, the more confident the prediction.



Three ways of building classifiers

- Generative

• Model $P(\tilde{x} | c_k)$

• Model $P(c_k)$

obtain $P(c_k | \tilde{x})$ using
Bayes Theorem

$$p(x, c = k) = p(x|c_k)p(c_k)$$

$$p(c_k|x) = \frac{p(x|c_k)p(c_k)}{p(x)}$$

Three ways of building classifiers

- Generative

- Model $P(\tilde{x} | c_k)$

- Model $P(c_k)$

- obtain $P(c_k | \tilde{x})$ using
Bayes Theorem

$$p(x, c = k) = p(x|c_k)p(c_k)$$

$$p(c_k|x) = \frac{p(x|c_k)p(c_k)}{p(x)}$$

- Discriminative

- Model $P(c_k | \tilde{x})$

Three ways of building classifiers

*probabilistic
classifiers*

- Generative

- Model $P(\tilde{x} | C_K)$
- Model $P(C_K)$
- obtain $P(C_K | \tilde{x})$ using Bayes Theorem

- Discriminative

- Model $P(C_K | \tilde{x})$

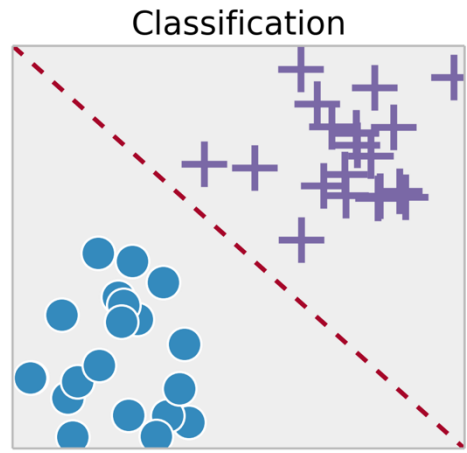
- Find decision boundaries

- Model $f: \tilde{x} \rightarrow K$

Can you see two natural ways to group these three approaches?

Probabilistic classifiers

- Probabilistic classifiers assign a probability that the input belongs to each class: $P(C_k | \mathbf{x})$
- If instead you just directly build a “scoring” function, $f(C_k, \mathbf{x})$, then you have no natural way to “understand” how confident the score is.
- Probabilistic classifiers always operate on the same “scale”, that of probability, which we can therefore more readily understand (and compare!).
- There are two main modelling approaches to obtain a probabilistic classifier.



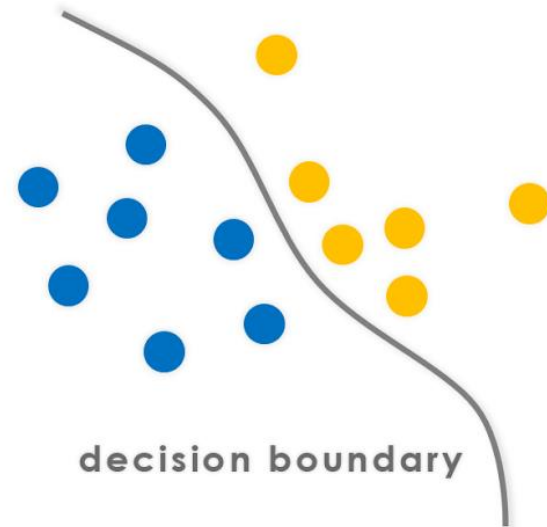
Two fundamental types of probabilistic classifiers

- Discriminative classifier vs. generative classifier.
- (Sometimes people just say “discriminative” vs. “generative”, but there are plenty of generative models which are not classifiers.)
- Many models with “discriminant” in the name are actually generative models (e.g. peak ahead: *linear discriminant analysis*)!
- (Also, logistic regression is doing classification not regression!)

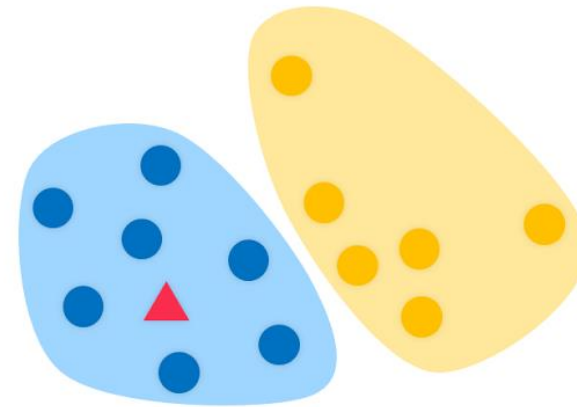
Discriminative vs Generative Classifiers

- What might these terms mean?
- Hint, look at these two pictures:

Discriminative



Generative



A tale of two children (discriminative vs generative)

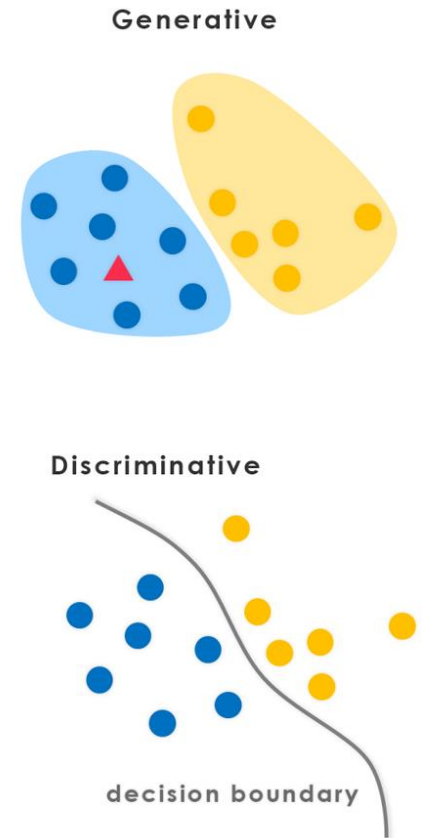
- A parent has two children, child A and child B.
- Child A has a knack for learning everything in a holistic manner.
- In contrast, Child B narrowly focuses on the most pertinent information needed, discarding the rest.
- The parent takes them to a tiny zoo with only lions and elephants, and tells the children to learn how to study the animals in order to be able to tell them apart.
- Later that night, they are watching a movie, and the parent asks the children: "is this animal we see on the screen a lion or an elephant?"

Can you think of the two different ways these children might have learned to build a lion-elephant classifier in their heads?

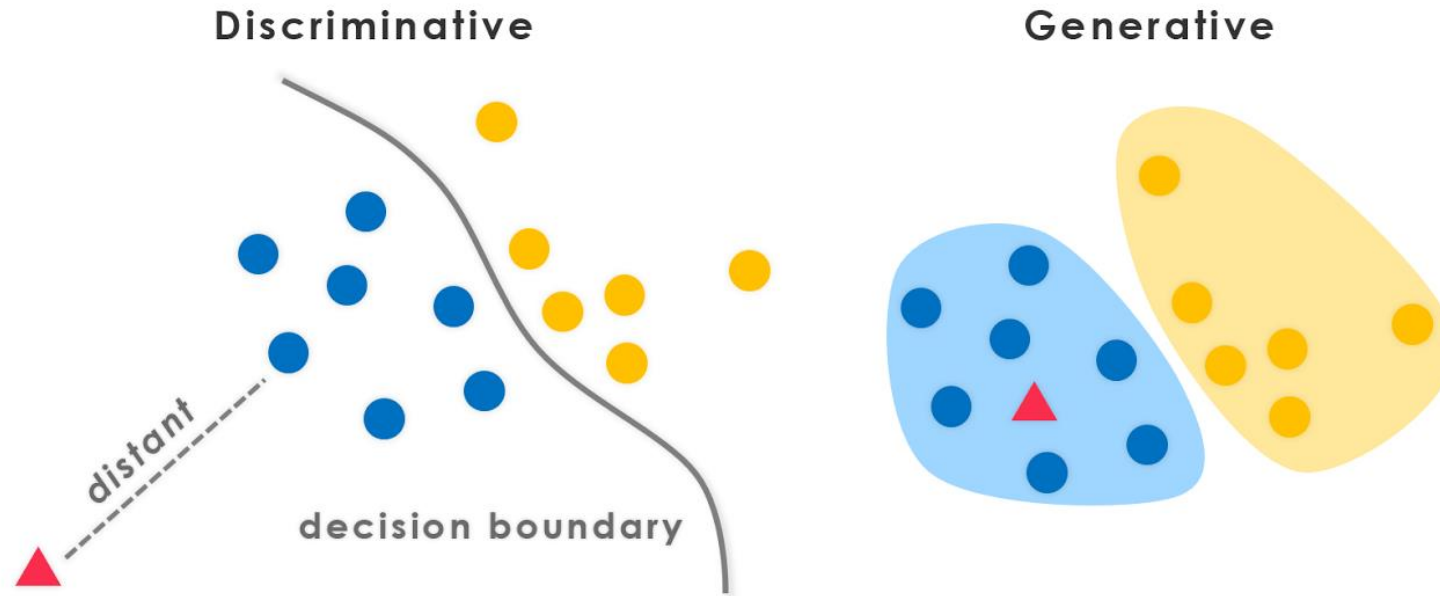
A tale of two children (discriminative vs generative)

Can you think of the two different ways these children might have learned to build a lion-elephant classifier in their heads?

- Child A had drawn two images, one of a lion and one of an elephant on a piece of paper. Then compares each image to the animal on the TV, and answers, "the animal is Lion".
- Child B can't remember what each animal looked like, but remembers the differences in some of their features (e.g. trunk or no trunk) and answers: "the animal is a Lion".
- Child A is "generative", vs. Child B who is "discriminative".



Discriminative vs Generative Classifiers



$$p(y|X)$$

$$\text{e.g., } p(y = C|X, w) = \text{sigmoid}(w^T x)$$

Logistic regression

$$p(y|X) \propto p(X, y) = p(X|y)p(y)$$

$$\text{e.g., } p(X|y = C, \mu_c, \Sigma_c) = N(X|\mu_c, \Sigma_c)$$

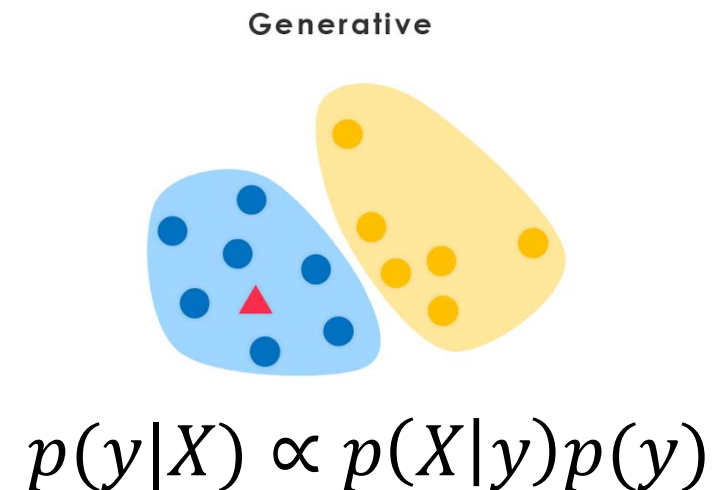
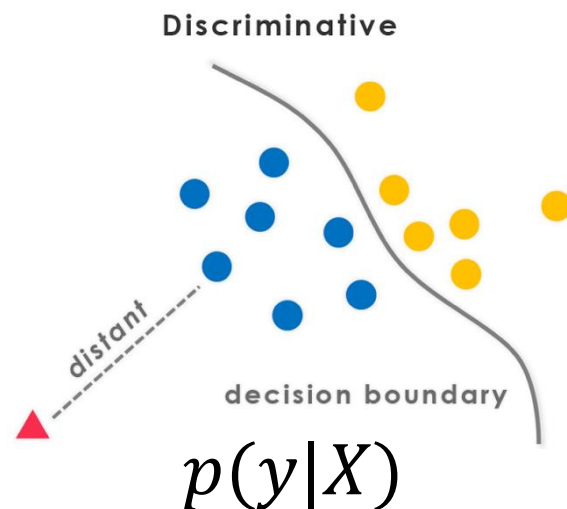
$$p(y = 1) = 0.22$$

$$p(y = 0) = 0.78$$

class-conditional gaussians

What implications does this have?

- Discriminative models “spend” all of their parameters to model the decision boundary.
- Generative models “spend” parameters to fully model fully model the density of $\mathbf{X}$ of each class, and imply a discriminative function.
- Thus, given the same capacity ($\approx$ # **params**), a discriminative model may yield more complex decision boundaries.



Discriminative vs Generative Classifiers

- A discriminative model directly models the discriminative function, $\text{score}(y = a) = f(X)$.
- This score may or may not be probabilistic.
- A generative model is defined by the joint probability, factored as:

$$p(y = C, X) = p(X|y = C)p(y = C).$$

- then uses Bayes Rule:

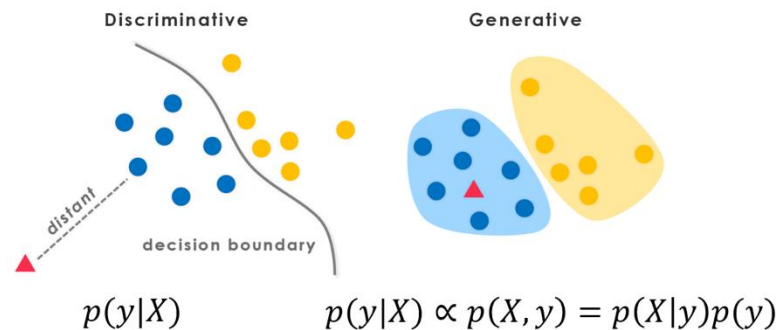
$$p(y = C|X) = \frac{p(X|y = C)p(y = C)}{p(X)}$$

Discriminative vs Generative Classifiers

- Generative model describes an (idealized) method of how the data were generated. In fact, can generate new pairs of data, $\{x_i, y_i\}$ from generative models.
- Generative models have discriminative properties (via Bayes Rule).
- Discriminative models do not have (full) generative properties. Could you generate data from them (e.g. Logistic Regression)?
- You can generate just the $\{y_i\}$ given the $\{x_i\}$.
- Sometimes unsupervised models, $p(x_i)$, are also called generative models (in fact, very commonly nowadays, even those which are not probabilistic, such as a GAN).

Discriminative vs Generative Classifiers

- Note that generative models make strong, explicit assumptions on statistical properties of $\mathbf{X}$ (e.g. class-conditional Gaussianity; independence among features, etc.), while discriminative models may only implicitly capture them.
- Thus discriminative models tend to be more robust/insensitive to model assumptions.
- But generative models can be more accurate when the data adhere to the model assumptions.

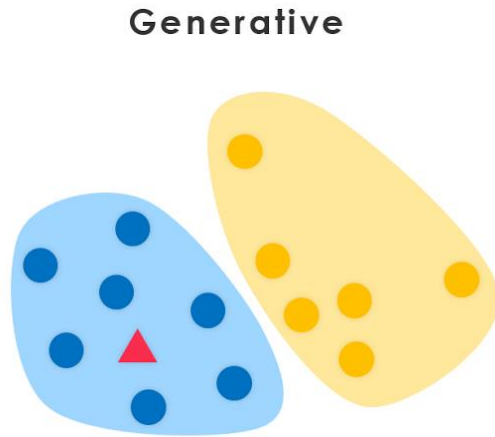


CS 189/289

Today's lecture outline

1. Discriminative vs. Generative Classification.
2. Gaussian Discriminant Analyses GDA.
3. ROC Curves

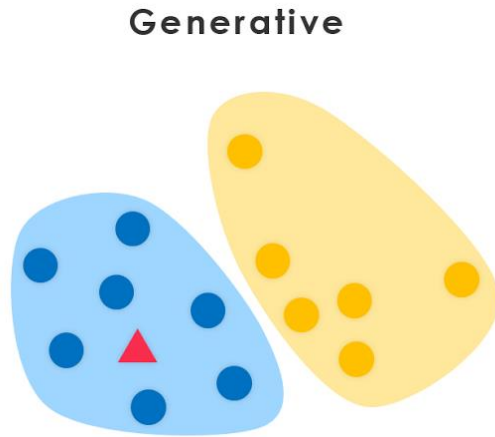
Gaussian “Discriminant” Analysis



- Generative model (not discriminative!)
- Use Gaussians to model X conditioned on the class.

$$p(X, y) = p(X|y)p(y)$$

Gaussian “Discriminant” Analysis



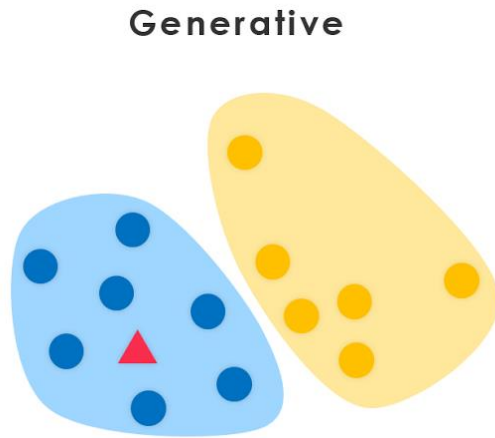
- Generative model.
- Use Gaussians to model X conditioned on the class.

$$p(X, y) = p(X|y)p(y)$$

$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$

$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

Gaussian “Discriminant” Analysis



- Generative model.
- Use Gaussians to model X conditioned on the class.

$$p(X, y) = p(X|y)p(y)$$

$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$

$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

$$p(y = 1) = \text{Bernoulli}(\phi) = 0.22$$

$$p(y = 0) = \text{Bernoulli}(1 - \phi) = 0.78$$

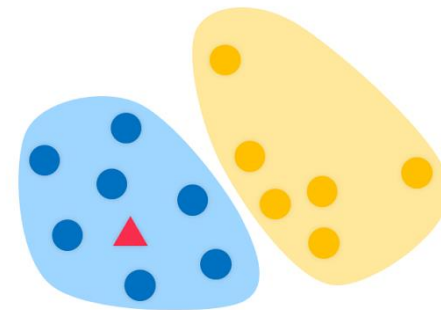
Gaussian Discriminant Analysis

What are the choices could we make:

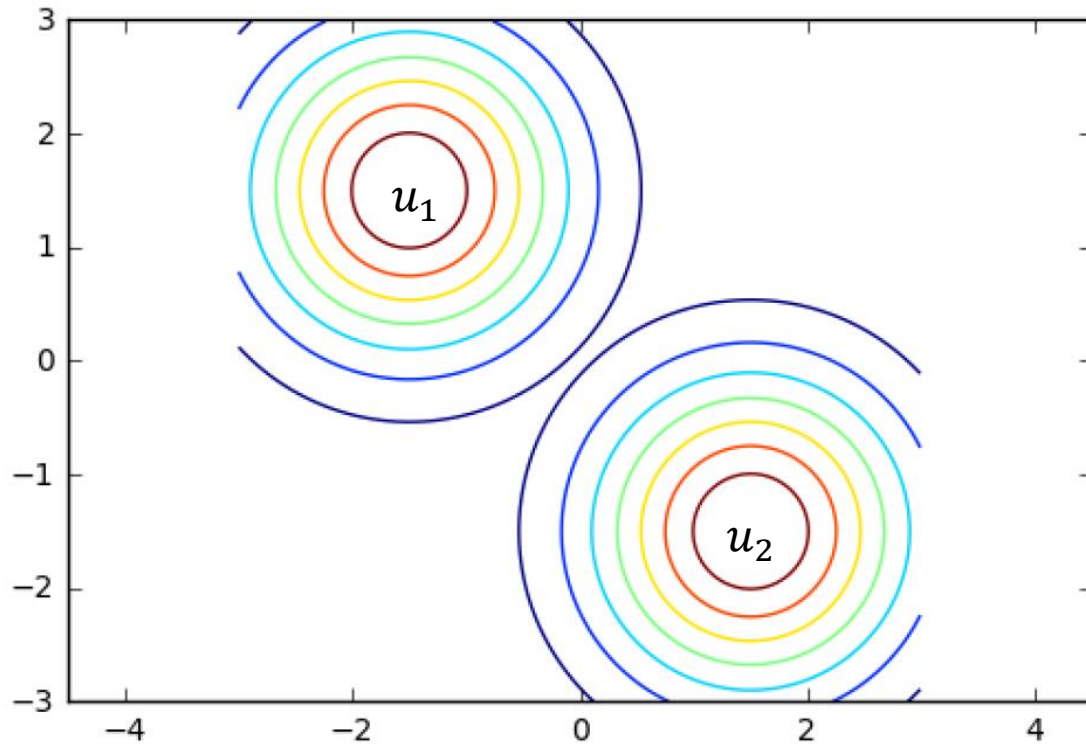
- Same/different mean parameters for each class?
- Same/different covariance parameters for each class?
- Should covariance be diagonal? Spherical?

$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$
$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

Generative



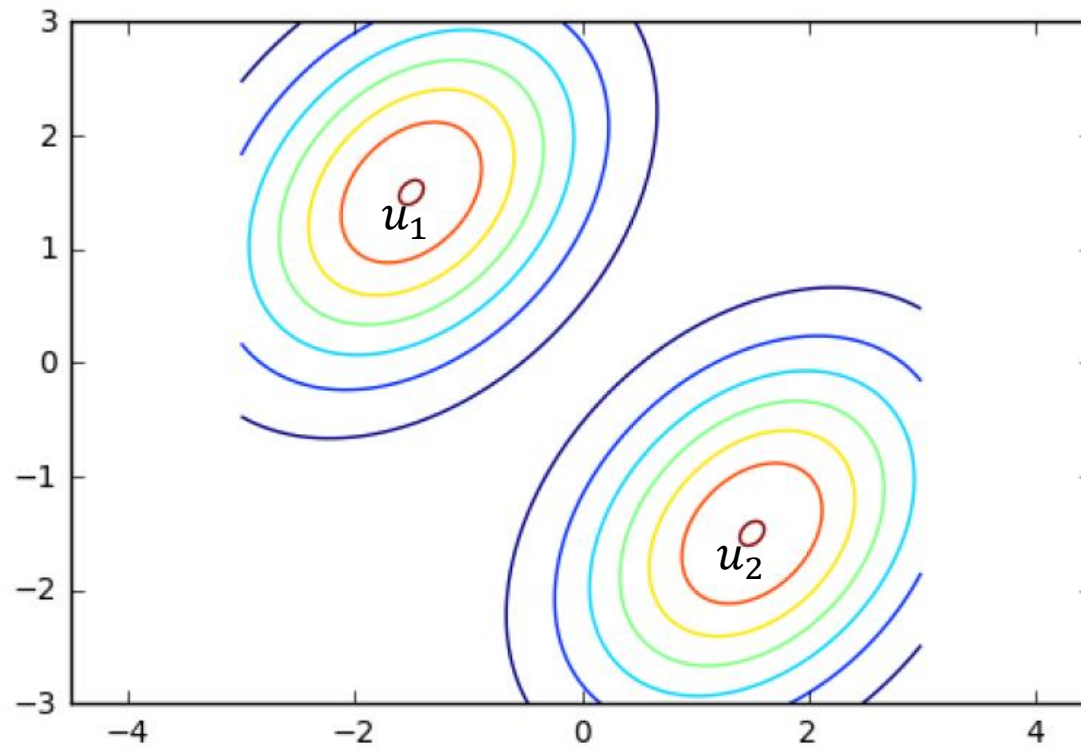
Gaussian Discriminant Analysis



We will always assume that we want different means, $\mathbf{u}_1$ and $\mathbf{u}_2$.

Two identical, “spherical” (isotropic) Gaussians.
i.e., $\Sigma_1 = \Sigma_2 = \Sigma = \text{diag}(\sigma, \sigma, \dots, \sigma)$

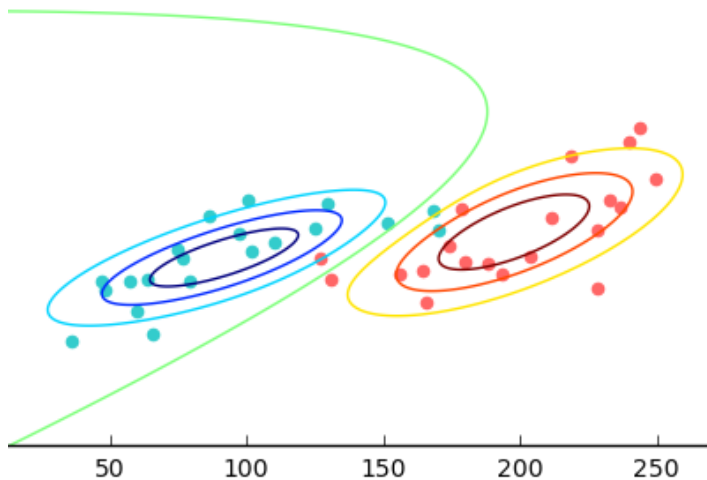
Gaussian Discriminant Analysis



Two identical, arbitrary covariance (anisotropic) Gaussians.
i.e., $\Sigma_1 = \Sigma_2 = \Sigma$

Gaussian Discriminant Analysis

Most general case: means and covariance can be different for each class, and covariance is arbitrary:



$$p(X|y = 1, \mu_1, \Sigma_1) = N(X|\mu_1, \Sigma_1)$$

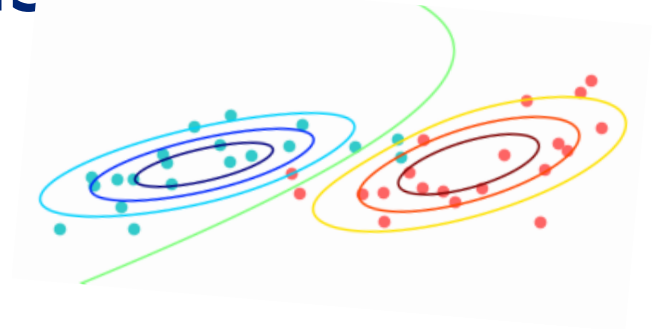
$$p(X|y = 0, \mu_0, \Sigma_0) = N(X|\mu_0, \Sigma_0)$$

Parameters: $\{\mu_0, \Sigma_0, \mu_1, \Sigma_1\}$

All of these variants readily generalize to $K > 2$ classes.

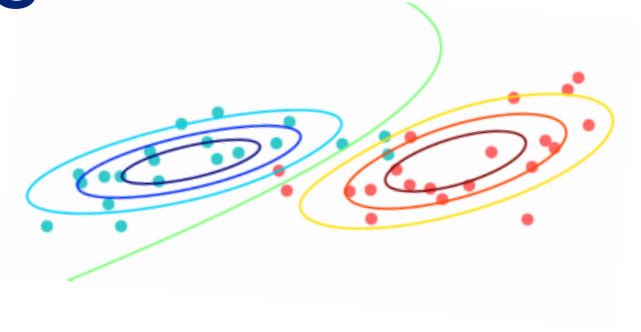
Gaussian Discriminant Analysis

- How to do parameter estimation?
- Maximum likelihood estimation (MLE).
- Data from k classes, $X \in R^{N \times D}$ and $Y \in \{1, \dots, K\}^N$.



Gaussian Discriminant Analysis

- How to do parameter estimation?
- Maximum likelihood.
- Data from k classes, $X \in R^{N \times D}$ and $Y \in \{1, \dots, K\}^N$.
- Conditioned on the class, the likelihood decouples into a Gaussian for data belonging to each class:



$$\log p(Y, X | \{\mu_k, \Sigma_k\}) = \sum_{\text{class } k} \sum_{i \in \{i | y_i = k\}} (\log N(x_i | \mu_k, \Sigma_k) + \log p(y_i))$$

this is how you know it is a generative model (and not a discriminative model)

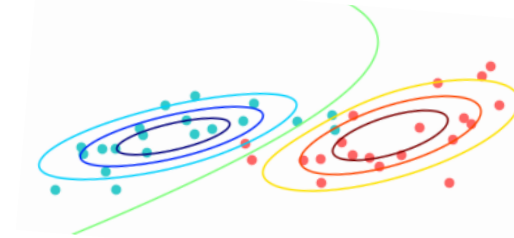
Gaussian Discriminant Analysis

What emerges is that parameter estimation decouples into MLE estimation of the Gaussian mean and covariance from the training data assigned to each class, k , separately.

Result:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{t_i=k} X_i$$

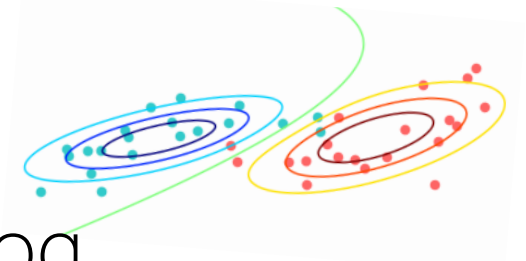
$$\hat{\Sigma}_k = \frac{1}{n_k} \sum_{t_i=k} (X_i - \hat{\mu}_k)(X_i - \hat{\mu}_k)^T$$



Log likelihood of the data:

$$\log p(Y, X | \{\mu_k, \Sigma_k\}) = \sum_{class\ k} \sum_{i \in \{i | y_i = k\}} (\log N(x_i | \mu_k, \Sigma_k) + \log p(y_i))$$

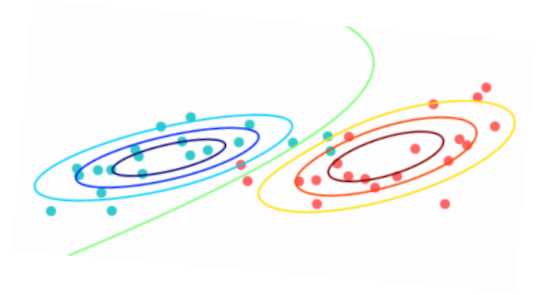
Gaussian Discriminant Analysis



These results can be justified by writing down the log likelihood, setting derivative to zero, etc.

For simplicity, let's go through the 1D derivation.

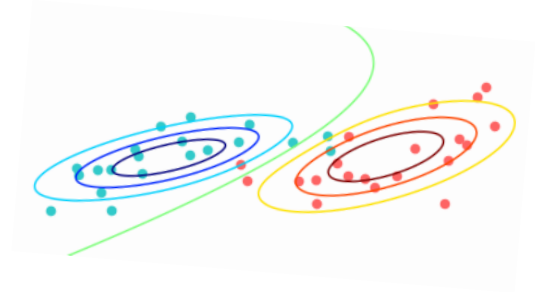
Gaussian Discriminant Analysis



1D derivation:

$$\hat{\mu}_k, \hat{\sigma}_k^2 = \arg \max_{\mu_k, \sigma_k^2} P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2)$$

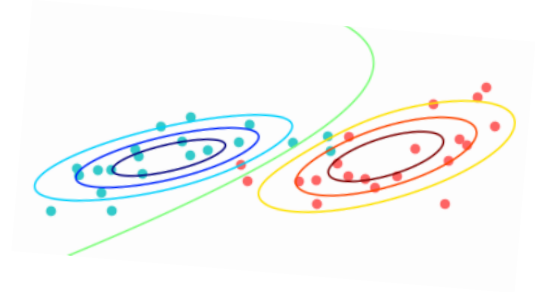
Gaussian Discriminant Analysis



1D derivation:

$$\begin{aligned}\hat{\mu}_k, \hat{\sigma}_k^2 &= \arg \max_{\mu_k, \sigma_k^2} P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2) \\ &= \arg \max_{\mu_k, \sigma_k^2} \ln(P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2))\end{aligned}$$

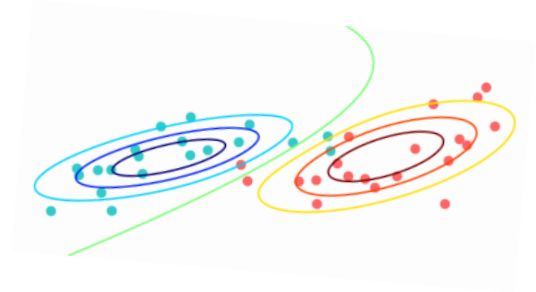
Gaussian Discriminant Analysis



1D derivation:

$$\begin{aligned}\hat{\mu}_k, \hat{\sigma}_k^2 &= \arg \max_{\mu_k, \sigma_k^2} P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2) \\ &= \arg \max_{\mu_k, \sigma_k^2} \ln(P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2)) \\ &= \arg \max_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} \ln(P(X_i | \mu_k, \sigma_k^2))\end{aligned}$$

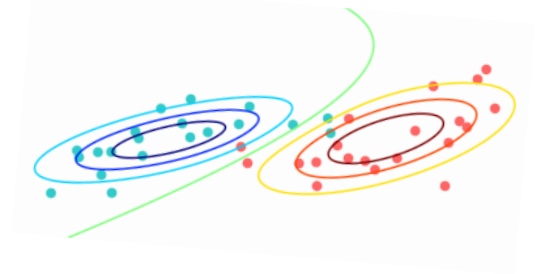
Gaussian Discriminant Analysis



1D derivation:

$$\begin{aligned}\hat{\mu}_k, \hat{\sigma}_k^2 &= \arg \max_{\mu_k, \sigma_k^2} P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2) \\ &= \arg \max_{\mu_k, \sigma_k^2} \ln(P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2)) \\ &= \arg \max_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} \ln(P(X_i | \mu_k, \sigma_k^2)) \\ &= \arg \max_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} -\frac{(X_i - \mu_k)^2}{2\sigma_k^2} - \ln(\sigma_k) - \frac{1}{2} \ln(2\pi)\end{aligned}$$

Gaussian Discriminant Analysis



1D derivation:

$$\begin{aligned}\hat{\mu}_k, \hat{\sigma}_k^2 &= \arg \max_{\mu_k, \sigma_k^2} P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2) \\&= \arg \max_{\mu_k, \sigma_k^2} \ln(P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2)) \\&= \arg \max_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} \ln(P(X_i | \mu_k, \sigma_k^2)) \\&= \arg \max_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} -\frac{(X_i - \mu_k)^2}{2\sigma_k^2} - \ln(\sigma_k) - \frac{1}{2} \ln(2\pi) \\&= \arg \min_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} \frac{(X_i - \mu_k)^2}{2\sigma_k^2} + \ln(\sigma_k)\end{aligned}$$

Gaussian Discriminant Analysis

1D derivation:

$$\hat{\mu}_k, \hat{\sigma}_k^2 = \arg \max_{\mu_k, \sigma_k^2} P(X_1, X_2, \dots, X_{n_k} | \mu_k, \sigma_k^2)$$

We need to minimize wrt both σ_k and μ_k --
can we get a closed form solution?

If they are not both dependent on one
another we might.

$$= \arg \min_{\mu_k, \sigma_k^2} \sum_{i=1}^{n_k} \frac{(X_i - \mu_k)^2}{2\sigma_k^2} + \ln(\sigma_k)$$

Gaussian Discriminant Analysis

Lets first try to solve for μ_k ,

$$\frac{\partial}{\partial \mu_k} \left(\sum_{i=1}^{n_k} \frac{(X_i - \mu_k)^2}{2\sigma_k^2} + \ln(\sigma_k) \right)$$

Gaussian Discriminant Analysis

Lets first try to solve for μ_k ,

$$\frac{\partial}{\partial \mu_k} \left(\sum_{i=1}^{n_k} \frac{(X_i - \mu_k)^2}{2\sigma_k^2} + \ln(\sigma_k) \right) = \sum_{i=1}^{n_k} \frac{-(X_i - \mu_k)}{\sigma_k^2} = 0$$

Gaussian Discriminant Analysis

Lets first try to solve for μ_k ,

$$\frac{\partial}{\partial \mu_k} \left(\sum_{i=1}^{n_k} \frac{(X_i - \mu_k)^2}{2\sigma_k^2} + \ln(\sigma_k) \right) = \sum_{i=1}^{n_k} \frac{-(X_i - \mu_k)}{\sigma_k^2} = 0$$

$$\implies \hat{\mu}_k = \frac{1}{n_k} \sum_{i=1}^{n_k} X_i$$

turns out that $\hat{\mu}_k$ does not depend on σ_k .

Gaussian Discriminant Analysis

Now lets try to solve for σ_k given $\hat{\mu}_k$:

$$\frac{\partial}{\partial \sigma} \left(\sum_{i=1}^{n_k} \frac{(X_i - \hat{\mu}_k)^2}{2\sigma_k^2} + \ln(\sigma_k) \right) = \sum_{i=1}^{n_k} -\frac{(X_i - \hat{\mu}_k)^2}{\sigma_k^3} + \frac{1}{\sigma_k} = 0$$

Gaussian Discriminant Analysis

Now lets try to solve for σ_k given $\hat{\mu}_k$:

$$\frac{\partial}{\partial \sigma} \left(\sum_{i=1}^{n_k} \frac{(X_i - \hat{\mu}_k)^2}{2\sigma_k^2} + \ln(\sigma_k) \right) = \sum_{i=1}^{n_k} -\frac{(X_i - \hat{\mu}_k)^2}{\sigma_k^3} + \frac{1}{\sigma_k} = 0$$

$$\Rightarrow \hat{\sigma}_k^2 = \frac{1}{n_k} \sum_{i=1}^{n_k} (X_i - \hat{\mu}_k)^2$$

(which depends on our estimate $\hat{\mu}_k$)

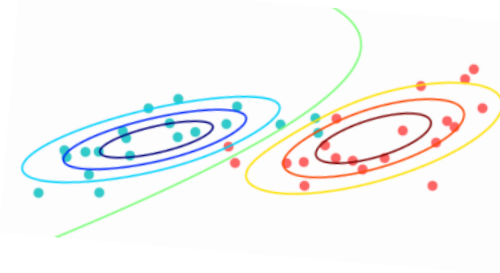
Gaussian Discriminant Analysis

Recap so far:

$$\log p(Y, X | \{\mu_k, \Sigma_k\}) = \sum_{\text{class } k} \sum_{i \in \{i | y_i = k\}} (\log N(x_i | \mu_k, \Sigma_k) + \log p(y_i))$$

$$\Rightarrow \hat{\mu}_k = \frac{1}{n_k} \sum_{t_i=k} X_i$$

$$\hat{\Sigma}_k = \frac{1}{n_k} \sum_{t_i=k} (X_i - \hat{\mu}_k)(X_i - \hat{\mu}_k)^T$$



What about setting $p(Y)$?

Recall: $p(Y|X) \propto p(X|Y)p(Y)$

Gaussian Discriminant Analysis $p(Y|X) \propto p(X|Y)p(Y)$

Can think of $p(y = \mathcal{C})$ in two ways:

- a) Compute empirically (estimate Bernoulli parameter):
MLE amounts to estimating what fraction of the training data belongs to each class, and setting the (generalized) Bernoulli parameters to those values.
- b) As a prior, specified by domain expert/knowledge (e.g. doctor who knows prevalence of Crohn's disease).

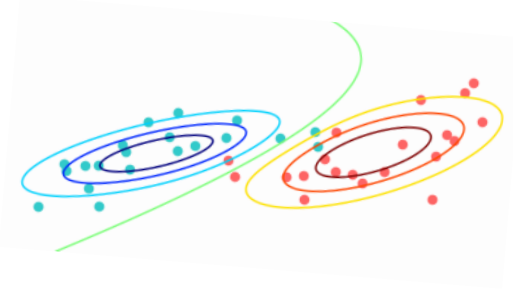
Why do (b) instead of (a) if you have training data?

Gaussian Discriminant Analysis $p(Y|X) \propto p(X|Y)p(Y)$

Now we know how to set all three kinds of parameters:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{t_i=k} X_i$$

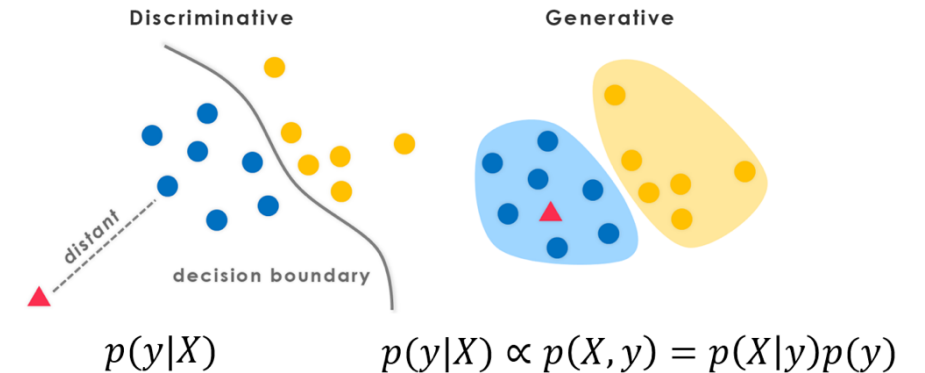
$$\hat{\Sigma}_k = \frac{1}{n_k} \sum_{t_i=k} (X_i - \hat{\mu}_k)(X_i - \hat{\mu}_k)^T$$



$p(Y)$ is learned from data, or set as a prior

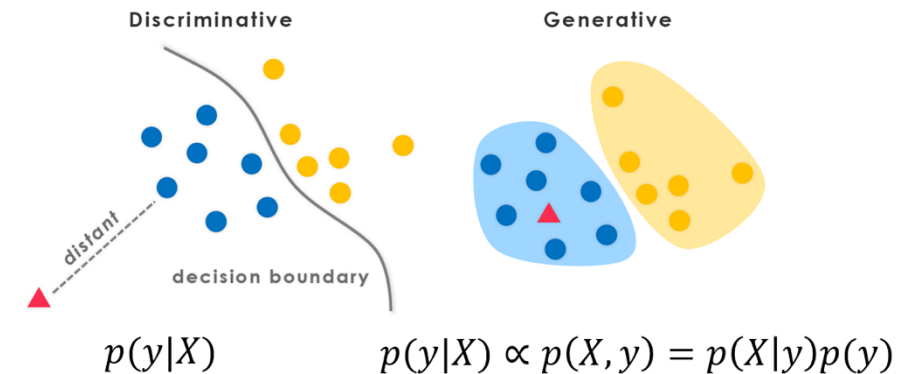
Gaussian Discriminant Analysis

- Lets take a closer look at the decision boundary implied by GDA.
- Formally, the decision boundary between class a and b is defined as ...?



Gaussian Discriminant Analysis

- Lets take a closer look at the decision boundary implied by GDA.
- Formally, the decision boundary between class a and b is defined as...
- ... the level set in X where $p(Y = a|X) = p(Y = b|X)$ for random variables $X \in R^{D \times 1}$ and $Y \in R^1$
- Lets use this equality to better understand the decision boundaries in GDA.

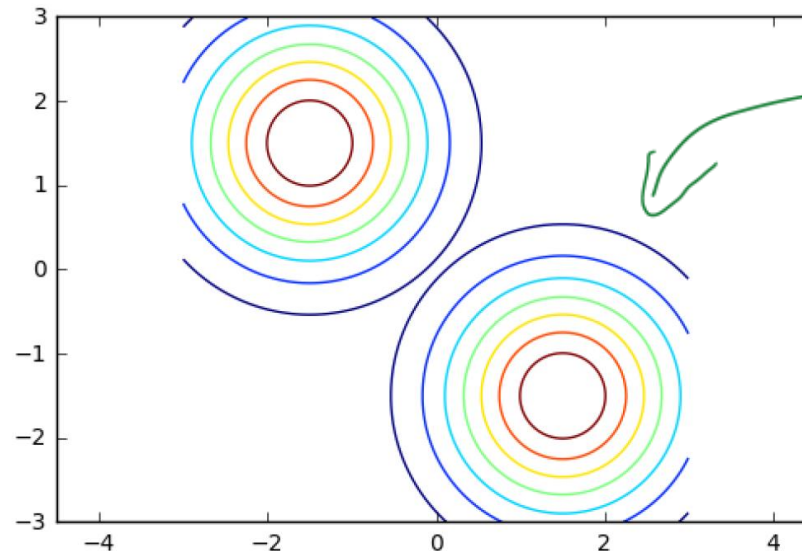


GDA Decision Boundary

- Lets assume equal class priors for each class ($p(a) = p(b)$).

GDA Decision Boundary

- Lets assume equal class priors for each class ($p(a) = p(b)$).
- Lets assume identical, isotropic (spherical, iid) Gaussian covariance for $p(X|Y)$ for each class: $\Sigma_A = \Sigma_B = \sigma^2 I$.



GDA Decision Boundary

Set class conditionals to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

GDA Decision Boundary

Set class conditionals to be the same:

$$p(Y = a|X) = p(Y = b|X)$$
$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$

GDA Decision Boundary

Set class conditionals to be the same:

$$\begin{aligned} p(Y = a|X) &= p(Y = b|X) \\ p(X|Y = a)p(Y = a) &= p(X|y = b)p(y = b) \\ p(X|Y = a) &= p(X|y = b) \end{aligned}$$

GDA Decision Boundary

Set class conditionals to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$

$$p(X|Y = a) = p(X|y = b)$$

$$N(X|\hat{\mu}_a, I\sigma^2) = N(X|\hat{\mu}_b, I\sigma^2)$$

GDA Decision Boundary

Set class conditionals to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$

$$p(X|Y = a) = p(X|y = b)$$

$$N(X|\hat{\mu}_a, I\sigma^2) = N(X|\hat{\mu}_b, I\sigma^2)$$

$$\log N(X|\hat{\mu}_a, I\sigma^2) = \log N(X|\hat{\mu}_b, I\sigma^2)$$

GDA Decision Boundary

Set class conditionals to be the same:

$$p(Y = a|X) = p(Y = b|X)$$

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$

$$p(X|Y = a) = p(X|y = b)$$

$$N(X|\hat{\mu}_a, I\sigma^2) = N(X|\hat{\mu}_b, I\sigma^2)$$

$$\log N(X|\hat{\mu}_a, I\sigma^2) = \log N(X|\hat{\mu}_b, I\sigma^2)$$

$$\begin{aligned} \longrightarrow & \ln\left(\frac{1}{2}\right) - \frac{1}{2}(X - \hat{\mu}_A)^T \sigma^2 I (X - \hat{\mu}_A) - \frac{1}{2} \ln(|\sigma^2 I|) \\ = & \ln\left(\frac{1}{2}\right) - \frac{1}{2}(X - \hat{\mu}_B)^T \sigma^2 I (X - \hat{\mu}_B) - \frac{1}{2} \ln(|\sigma^2 I|) \end{aligned}$$

GDA Decision Boundary

$$\begin{aligned} \cancel{\ln\left(\frac{1}{2}\right)} - \frac{1}{2}(X - \hat{\mu}_A)^T \sigma^2 I (X - \hat{\mu}_A) - \cancel{\frac{1}{2} \ln(|\sigma^2 I|)} \\ = \cancel{\ln\left(\frac{1}{2}\right)} - \frac{1}{2}(X - \hat{\mu}_B)^T \sigma^2 I (X - \hat{\mu}_B) - \cancel{\frac{1}{2} \ln(|\sigma^2 I|)} \end{aligned}$$

$$\Rightarrow \boxed{(X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)}$$

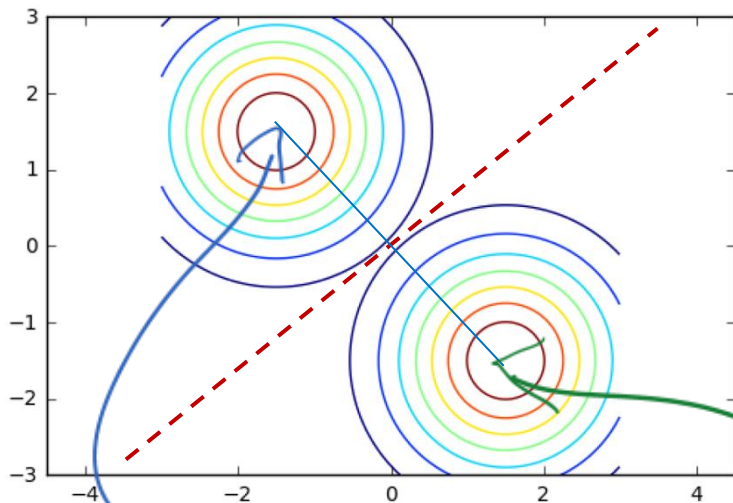
GDA Decision Boundary

$$\ln\left(\frac{1}{2}\right) - \frac{1}{2}(X - \hat{\mu}_A)^T \sigma^2 I (X - \hat{\mu}_A) - \frac{1}{2} \ln(|\sigma^2 I|)$$

Thus decision boundary is set of X for which

i.e. points equidistant from $\hat{\mu}_a$ and $\hat{\mu}_b$.

$$\Rightarrow (X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$$



Thus decision boundary is set of X for which $\|X - \hat{\mu}_A\|_2 = \|X - \hat{\mu}_B\|_2$,

i.e. points equidistant from $\hat{\mu}_a$ and $\hat{\mu}_b$.

GDA Decision Boundary

- We did not do the general case—we had assumed equal class priors and equal spherical covariances.
- What happens if we let the covariances be equal, but of arbitrary shape (anisotropic)?

GDA Decision Boundary

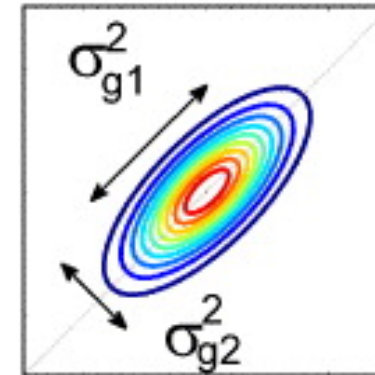
- In isotropic case, decision boundary was dictated by $(X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$ which led to a line.

GDA Decision Boundary

- In isotropic case, decision boundary was dictated by $(X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$ which led to a line.
- With arbitrary covariance, it is dictated by:
$$(X - \hat{\mu}_A)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_B)$$

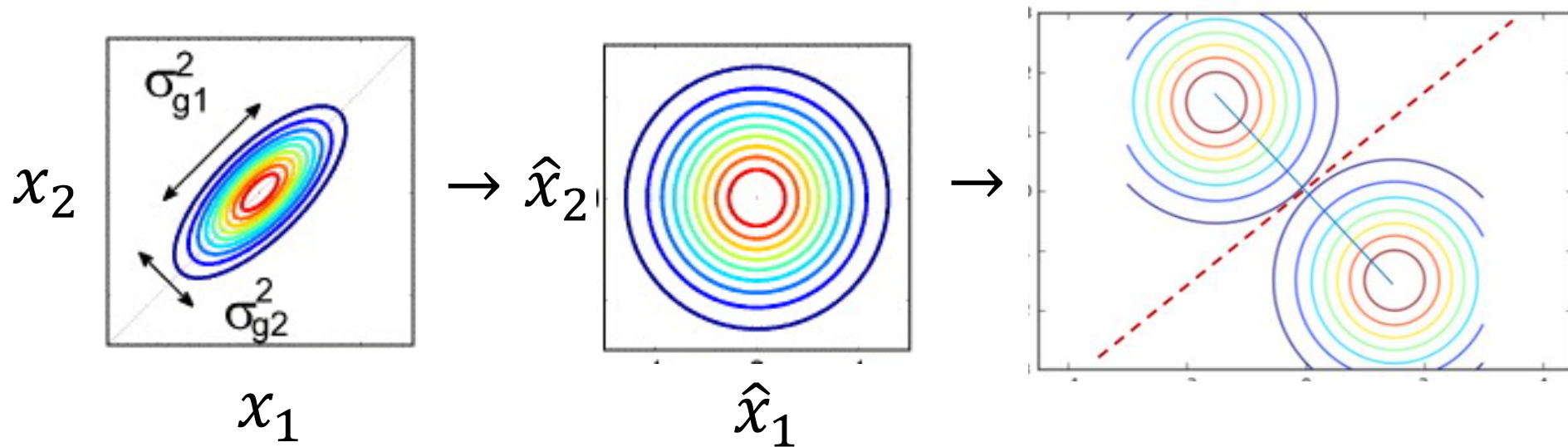
GDA Decision Boundary

- In isotropic case, decision boundary was dictated by $(X - \hat{\mu}_A)^T (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T (X - \hat{\mu}_B)$ which led to a line.
- With arbitrary covariance, it is dictated by:
$$(X - \hat{\mu}_A)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_A) = (X - \hat{\mu}_B)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_B)$$
- Each of these, $(X - \hat{\mu}_k)^T \hat{\Sigma}^{-1} (X - \hat{\mu}_k)$, is an ellipsoid—axes are shrunk and/or rotated.
- (Same for both classes).



GDA Decision Boundary

- As earlier in class, use $\Sigma = USU^T$ to find coordinate system $\hat{X} = S^{-\frac{1}{2}}U^T(X - \mu_A)$ to make $p(\hat{X})$ spherical.



Thus decision boundary is still a line.

GDA Decision Boundary

What happens if the class priors are not equal?

GDA Decision Boundary

What happens if the class priors are not equal?

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}) + \log p(Y = a) = \log N(X|\hat{\mu}_b, \hat{\Sigma}) + \log p(Y = b)$$

GDA Decision Boundary

What happens if the class priors are not equal?

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}) + \log p(Y = a) = \log N(X|\hat{\mu}_b, \hat{\Sigma}) + \log p(Y = b)$$

- We are adding a constant into the system.

GDA Decision Boundary

What happens if the class priors are not equal?

$$p(X|Y = a)p(Y = b) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}) + \log p(Y = b) = \log N(X|\hat{\mu}_b, \hat{\Sigma}) + \log p(Y = b)$$

- We are adding a constant into the system.
- Won't go through details, but leaves the decision boundary as a line.
- Think about intuitively why you might expect this.

GDA Decision Boundary

Finally, what happens if also allow different covariances for each class?

GDA Decision Boundary

Finally, what happens if also allow different covariances for each class?

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}_a) + \log p(Y = a) = \log N(X|\hat{\mu}_b, \hat{\Sigma}_b) + \log p(Y = b)$$

GDA Decision Boundary

Finally, what happens if also allow different covariances for each class?

$$p(X|Y = a)p(Y = a) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}_a) + \log p(Y = a) = \log N(X|\hat{\mu}_b, \hat{\Sigma}_b) + \log p(Y = b)$$

$$\ln(P(A)) - \frac{1}{2}(X - \hat{\mu}_A)^T \hat{\Sigma}_A^{-1} (X - \hat{\mu}_A) :$$
$$= \ln(P(B)) - \frac{1}{2}(X - \hat{\mu}_B)^T \hat{\Sigma}_B^{-1} (X - \hat{\mu}_B)$$

GDA Decision Boundary

Finally, what happens if also allow different covariances for each class?

$$p(X|Y = a)p(Y = b) = p(X|y = b)p(y = b)$$
$$\log N(X|\hat{\mu}_a, \hat{\Sigma}_a) + \log p(Y = b) = \log N(X|\hat{\mu}_b, \hat{\Sigma}_b) + \log p(Y = b)$$

$$\ln(P(A)) - \frac{1}{2}(X - \hat{\mu}_A)^T \hat{\Sigma}_A^{-1} (X - \hat{\mu}_A) :$$
$$= \ln(P(B)) - \frac{1}{2}(X - \hat{\mu}_B)^T \hat{\Sigma}_B^{-1} (X - \hat{\mu}_B)$$

Cannot get rid of quadratic terms, so decision boundary is a quadratic in X .

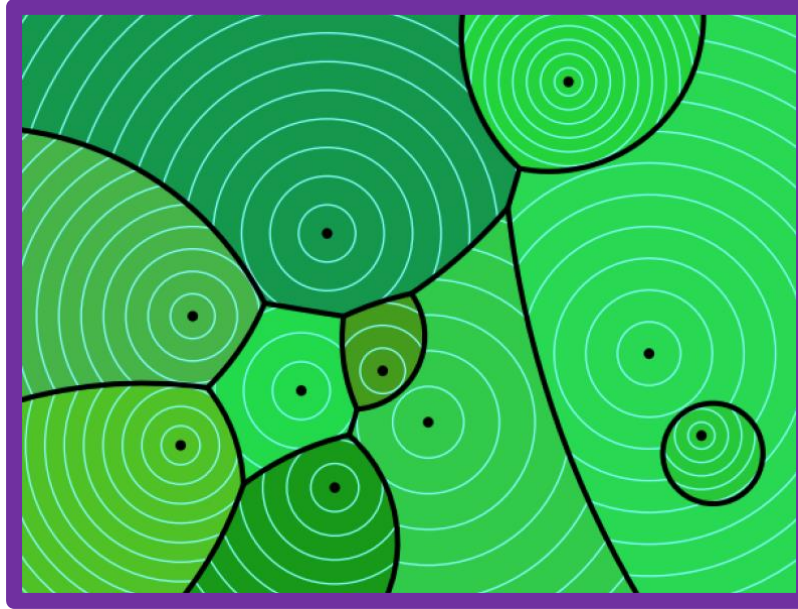
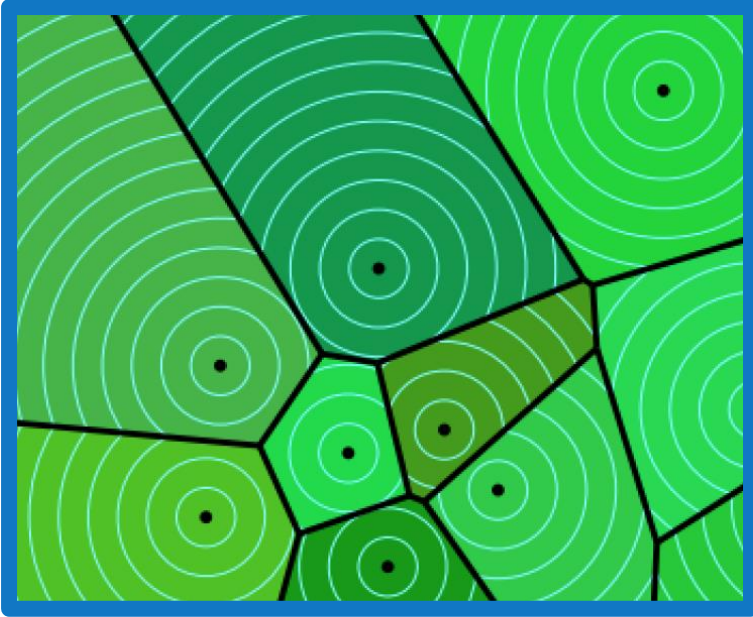
Summary of GDA Decision Boundaries

- When class covariances are equal, decision boundary is linear: *Linear Discriminant Analysis* (LDA).
- When class covariances are unequal, decision boundary is quadratic: called *Quadratic Discriminant Analysis* (QDA).
- All these results also hold for multi-class (more than 2 classes).

LDA

vs

QDA



- In general, QDA uses an extra $\frac{(K-1)D^2}{2}$ extra parameters to define the covariance matrices for each class (beyond LDA).
- This could hurt us because of bias/variance trade-off.

Predictive distribution for LDA is...?

- Desired predictive conditional is given by:

$$\begin{aligned} p(Y = a|X) &\propto p(X|Y = a)p(Y = a) \\ &= N(\mu_a, \Sigma)p(Y = a) \end{aligned}$$

Predictive distribution for LDA is...?

- Desired predictive conditional is given by:

$$\begin{aligned} p(Y = a|X) &\propto p(X|Y = a)p(Y = a) \\ &= N(\mu_a, \Sigma)p(Y = a) \end{aligned}$$

- This is the “discriminative” probability derived from the generative model, by way of Bayes Rule that Prof. Malik also showed you.
- He also showed you that it takes the form of logistic regression:

$$p(Y = a|X) = \frac{1}{1 + \exp(-w^T X)}$$

- Wait—did we go through all of that just to get to logistic regression?
- Is GDA a discriminative or generative model?!

LDA vs Logistic regression

1. LDA model can always be written as a logistic regression (LR) model (Prof. Malik showed).
2. However, the converse is not true! Not every logistic regression model can be written as an LDA model.
3. Thus LDA makes strictly stronger modelling assumptions than LR, and LR is a more general model. (More general here means that it can trace out more complicated decision boundaries).

LDA vs Logistic regression

4. When LDA assumptions are correct (class-conditional densities are Gaussian), LDA tends to be better than LR.
5. As data goes to infinity, results more similar.
6. This is generally true for generative vs. discriminative models:

When assumptions of generative model are met, it can be a better model than discriminative counterpart, presumably because restricting the space.

What about when features, x_i are discrete?

- The “density” part of the generative model, $p(X|Y = a)$ becomes a discrete distribution.
- If we assume that all of the features are independent, $p(X|Y = a) = \prod_{f=1}^D p(X_f|Y = a)$ then we get the widely known “Naïve Bayes” model.
- It is naïve because of the assumption on independence of features (analogous to spherical Gaussian).
- Can be augmented in various ways to mitigate this assumption, e.g. with “tree-augmented Bayes”.